

Pi Developer Architecture Guide

A source-grounded path from first contribution to extension authoring

Pi maintainers

2026-07-09

Table of contents

1	Welcome to the Pi codebase	8
1.1	What Pi is	8
1.2	A mental model in one diagram	8
1.3	Reading order and outcomes	9
1.4	Source map used for this guide	9
2	TypeScript used by Pi	11
2.1	Modules and imports	11
2.2	Types describe values; they do not validate values	12
2.3	Narrowing unions with a discriminant	12
2.4	Generics: preserve a relationship between inputs and outputs	13
2.5	<code>unknown</code> is the honest type for untrusted data	13
2.6	Async code and cancellation	13
2.7	Objects, spreads, and immutable snapshots	14
2.8	Functions are values and callbacks define lifecycle seams	14
2.9	Classes and public fields	15
2.10	TypeBox: runtime schemas for tool inputs	15
2.11	A compact reading checklist	15
3	Architecture: packages, ownership, and entry points	17
3.1	Dependency direction	17
3.2	Repository-level conventions	17
3.3	Component walkthrough	18
3.3.1	<code>packages/tui</code> : terminal rendering primitives	18
3.3.2	<code>packages/ai</code> : a provider-neutral streaming contract	18
3.3.3	<code>packages/agent</code> : generic conversation and tool loop	18
3.3.4	<code>packages/coding-agent</code> : the Pi application	19
3.3.5	<code>packages/orchestrator</code> : experimental process orchestration	20
3.4	Constructing a running coding-agent session	20
3.5	Persistence and context are different things	20
3.6	Where to make a change	21
4	Prompt lifecycle: from Enter to the next render	22
4.1	The complete path	22
4.2	Stage 1: Terminal input enters the TUI	22
4.3	Stage 2: Commands, keybindings, and input interception	23
4.4	Stage 3: Agent-session orchestration constructs a turn	23
4.5	Stage 4: Context and system prompt are built at two boundaries	23
4.6	Stage 5: Model and provider selection	24
4.7	Stage 6: Streaming response becomes agent events and TUI updates	24
4.8	Stage 7: Tool calls are parsed, validated, gated, and executed	25

4.9	Stage 8: Results create the next agent action	25
4.10	Stage 9: Persistence and rendering settle the turn	26
4.11	Stage 10: Errors, cancellation, retries, compaction, and shutdown	26
4.12	Trace this yourself	26
5	Extensions: change Pi without forking it	28
5.1	Choose the right customization mechanism	28
5.2	Discovery, trust, and loading	28
5.3	The lifecycle you can rely on	29
5.3.1	Event groups and common use	29
5.4	A complete first extension	30
5.5	What <code>ExtensionAPI</code> and context provide	31
5.6	Tools: make the model contract explicit	32
5.7	Commands, UI, and rendering	33
5.8	State, sessions, and queues	33
5.9	Distribute an extension as a Pi package	33
5.10	Debugging extensions	34
6	Customization patterns and contributor workflow	35
6.1	Configuration before code	35
6.2	Pattern: a safety gate around a built-in tool	35
6.3	Pattern: inject focused context, not a second system	36
6.4	Pattern: state that survives restart and branching	36
6.5	Pattern: dynamic tools and active-tool policy	37
6.6	Pattern: provider customization	37
6.7	Pattern: custom TUI that still works outside the TUI	37
6.8	Pattern: package a team customization	38
6.9	Common mistakes and the correct boundary	38
6.10	A practical verification ladder	38
7	Sessions, branches, and durable context	40
7.1	Why persistence is a tree	40
7.2	Files, headers, and locations	40
7.3	The entry vocabulary	41
7.4	Message shapes at the provider boundary	41
7.5	<code>SessionManager</code> as the persistence boundary	42
7.6	Building context from a selected leaf	42
7.7	Compaction is a controlled replacement of old context	43
7.7.1	Split turns and tool continuity	43
7.8	Branch navigation and branch summaries	44
7.9	Session replacement is not a mutation of the old runtime	44
7.10	Session names, labels, and human navigation	44
7.11	Extension persistence patterns	45
7.11.1	Stateful tool results	45
7.11.2	Private durable data	45
7.11.3	Persistent model context	45
7.12	Session debugging checklist	45

8	Resources, settings, and project trust	46
8.1	The startup services boundary	46
8.2	Global and project settings	46
8.3	Trust is an execution policy, not a cosmetic prompt	47
8.4	Resource categories	47
8.5	Explicit resource paths and patterns	48
8.6	Skills and prompt templates on the input path	48
8.7	Context files and system-prompt construction	48
8.8	Settings that alter runtime behavior	49
8.8.1	Model and thinking	49
8.8.2	Compaction and branch summary	49
8.8.3	Retry and delivery	49
8.8.4	Shell, images, and terminal	49
8.9	Project configuration that an extension should respect	50
8.10	Packages are resource delivery, not a new execution mode	50
8.11	Themes and the TUI boundary	50
8.12	Reload semantics	50
8.13	Configuration-review checklist	51
9	Built-in tools: filesystem and shell execution	52
9.1	Tool registry and default selection	52
9.2	Shared design principles	52
9.3	Output limits protect the next provider turn	53
9.4	read : text ranges and image attachments	53
9.4.1	Read operations are pluggable	53
9.5	write : create or replace a complete file	54
9.6	edit : exact replacement and queued mutation	54
9.7	bash : process execution with streamed tail output	54
9.7.1	Streaming output	55
9.8	grep , find , and ls : bounded repository discovery	55
9.9	Paths, cwd, and remote operations	56
9.10	User shell commands are separate from model bash	56
9.11	Tool rendering is a first-class interface	56
9.12	Tool lifecycle and extension interception	57
9.13	A safe custom file-mutation skeleton	57
9.14	Tool-author review questions	57
10	Models, providers, streaming, and authentication	59
10.1	The Model contract	59
10.2	Model registry assembly	59
10.3	Selecting an initial model	60
10.4	Thinking levels are Pi vocabulary	60
10.5	Custom models in models.json	61
10.6	Provider overrides and model overrides	61
10.7	Config values and secrets	62
10.8	Credential precedence and storage	62
10.9	Provider headers and request layering	62
10.10	The unified streaming API	63
10.11	Payload and response hooks	63

10.12	Transport, timeout, and retry	63
10.13	Custom providers from an extension	63
10.14	Provider debugging workflow	64
11	Agent core: runs, events, queues, and tool batches	65
11.1	The three layers inside the package	65
11.2	Agent state and snapshots	65
11.3	Starting a prompt versus continuing a context	66
11.4	Active-run lifecycle and cancellation	66
11.5	Event stream as the observable protocol	66
11.6	One provider turn in detail	67
11.7	Tool argument preparation and validation	67
11.8	Sequential and parallel tool execution	67
11.9	Before and after tool hooks	68
11.10	Termination, steering, and follow-up	68
11.11	Next-turn refresh hook	68
11.12	Error semantics are intentionally normalized	69
11.13	Embedding the agent package	69
11.14	Agent-core debugging checklist	69
12	Terminal UI and interactive mode	70
12.1	The TUI package owns terminal mechanics	70
12.2	Differential rendering in practical terms	70
12.3	Components are composition, not application policy	71
12.4	Terminal lifecycle and capabilities	71
12.5	Keyboard input is parsed before it becomes a binding	71
12.6	Editor submission and queue semantics	72
12.7	Transcript rendering from agent events	72
12.8	Tools in the transcript	72
12.9	Extension UI surface	73
12.10	Custom components and focus	73
12.11	Autocomplete composition	74
12.12	Themes and semantic styling	74
12.13	Images and rendered width	74
12.14	Debugging visual issues	75
13	Testing, debugging, and contribution workflow	76
13.1	Start with a reproducible boundary	76
13.2	Static checking versus tests	76
13.3	Test at the correct package	77
13.4	High-value test cases by subsystem	77
13.4.1	Extensions	77
13.4.2	Tools	77
13.4.3	Sessions and compaction	78
13.4.4	Provider/model registry	78
13.5	Interactive TUI smoke testing	78
13.6	/debug and diagnostic data	78
13.7	Error classification before retrying	78
13.8	Source navigation habits	79

13.9	Git hygiene in a shared worktree	79
13.10	Documentation as a tested interface	79
13.11	Pre-review checklist	80
14	Guided source tours and architecture exercises	81
14.1	Tour 1: What launches when <code>pi</code> starts?	81
14.2	Tour 2: Why can a session change <code>cwd</code> safely?	81
14.3	Tour 3: How do resources reach a system prompt?	82
14.4	Tour 4: What happens to a user message before a provider sees it?	82
14.5	Tour 5: Follow one streamed token to the terminal	83
14.6	Tour 6: Follow one tool call through validation	83
14.7	Tour 7: Study parallel tool execution without guessing	84
14.8	Tour 8: Explain a model request without opening a network trace	84
14.9	Tour 9: Trace a resume-context discrepancy	84
14.10	Tour 10: Add a narrowly scoped extension feature	85
14.11	A weekly reading routine for new contributors	85
15	Extension event and API reference	86
15.1	Choose an event by the representation you need	86
15.2	Startup and trust events	87
15.2.1	<code>project_trust</code>	87
15.2.2	<code>resources_discover</code>	87
15.3	Session events	87
15.3.1	<code>session_start</code> and <code>session_shutdown</code>	87
15.3.2	<code>session_before_switch</code> and <code>session_before_fork</code>	87
15.3.3	Compaction and tree hooks	88
15.3.4	Session information	88
15.4	Input and agent-run events	88
15.4.1	<code>input</code>	88
15.4.2	<code>before_agent_start</code>	88
15.4.3	<code>agent_start</code> , <code>agent_end</code> , and <code>agent_settled</code>	89
15.4.4	<code>turn_start</code> , <code>turn_end</code> , and messages	89
15.5	Provider events	89
15.5.1	<code>before_provider_headers</code>	89
15.5.2	<code>before_provider_request</code>	89
15.5.3	<code>after_provider_response</code>	89
15.6	Tool events and parallelism	90
15.7	Extension context reference	90
15.8	Command context reference	91
15.9	Registration API quick decisions	91
15.10	Safety rules for extension authors	91
16	Run modes, SDK embedding, and operational boundaries	93
16.1	How Pi chooses a mode	93
16.2	Extension mode behavior	93
16.3	Print mode as a single-run host	94
16.4	JSON mode and machine-readable automation	94
16.5	RPC mode as a persistent process boundary	94
16.6	The public SDK entry point	95

16.7 Why SDK callers should not reimplement <code>main.ts</code>	95
16.8 Custom tools supplied by an SDK host	95
16.9 Initial messages and attachments	96
16.10 Operational safety boundaries	96
16.11 Operational observability	96
16.12 Choosing an integration approach	97
16.13 Operational checklist	97
17 Failure and recovery reference	98
17.1 Failure flow by runtime layer	98
17.2 Startup and configuration failures	98
17.3 Provider failures	99
17.4 Tool failures	99
17.5 Cancellation versus timeout	100
17.6 Recovery after compaction and tree movement	100
17.7 UI and renderer fallback	100
17.8 Incident checklist	101
18 Glossary	102

1 Welcome to the Pi codebase

This guide is for a developer who can read Python and JavaScript, but does not yet use TypeScript every day. Its job is practical: by the end, you should be able to trace a prompt through Pi, find the right layer for a change, and build an extension without guessing at the API.

This is a guide to the repository as checked out, not a replacement for the published user documentation. File paths throughout are repository-relative. They are deliberately specific so that you can open the implementation beside this book. When a section simplifies a control flow, it says so; the named source files are the authority for edge cases.

1.1 What Pi is

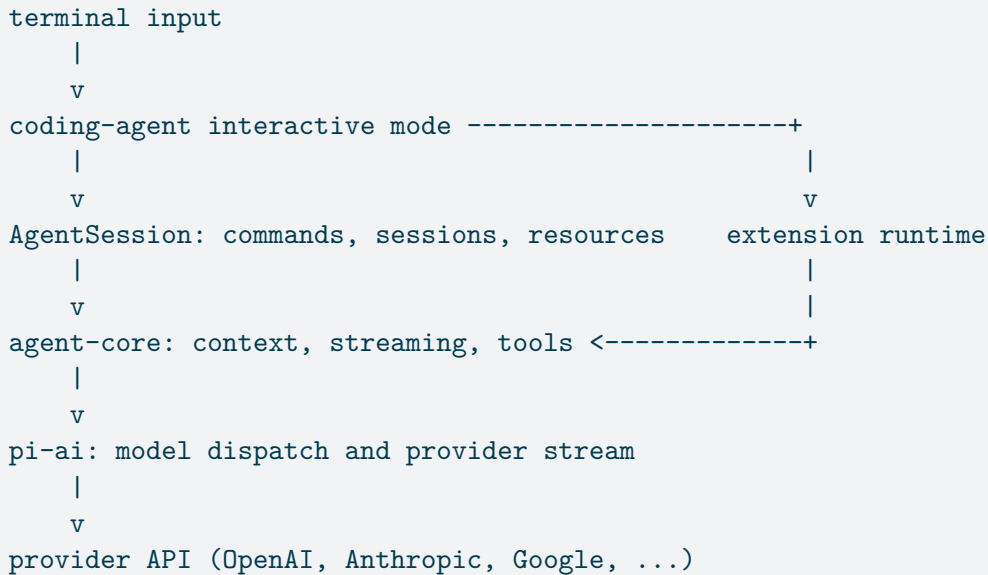
Pi is a TypeScript monorepo for an agent harness and a self-extensible coding agent. The root package is an npm workspace. The interactive `pi` command is implemented by `@earendil-works/pi-coding-agent`; it composes a terminal UI, an agent runtime, and a provider-neutral AI API.

The four primary packages are:

Package	Role	Start reading at
<code>packages/tui</code>	Reusable terminal UI primitives and differential rendering	<code>src/index.ts</code> , <code>src/tui.ts</code>
<code>packages/ai</code>	Models, provider dispatch, streaming API adapters, authentication primitives	<code>src/index.ts</code> , <code>src/types.ts</code> , <code>src/compat.ts</code>
<code>packages/agent</code>	Provider-independent agent loop, message state, tool execution	<code>src/agent.ts</code> , <code>src/agent-loop.ts</code> , <code>src/types.ts</code>
<code>packages/coding-agent</code>	CLI, interactive application, sessions, built-in tools, configuration, extensions	<code>src/main.ts</code> , <code>src/core/sdk.ts</code> , <code>src/core/agent-session.ts</code>

`packages/orchestrator` is an experimental package depending on the coding agent. It is useful when you specifically need its process orchestration surface, but it is not on the normal interactive prompt path described here.

1.2 A mental model in one diagram



The arrows point in the usual runtime direction. The extension runtime has hooks at several boundaries; it does not bypass the agent loop merely because it can add tools or alter a request.

1.3 Reading order and outcomes

1. [TypeScript used by Pi](#) makes the type syntax and module conventions readable.
2. [Architecture](#) maps packages, ownership, persistence, and runtime entry points.
3. [Prompt lifecycle](#) follows one prompt through every major runtime boundary.
4. [Extensions](#) explains discovery, hooks, tools, UI, state, packaging, and a complete minimal extension.
5. [Customization and workflow](#) helps you choose an extension, configuration, SDK use, or core change and validates your work safely.

The [glossary](#) is a quick reference for terms that recur across the codebase.

1.4 Source map used for this guide

The guide was checked against the package manifests, the extension examples, the coding-agent extension documentation, and these implementation seams:

- CLI composition: `packages/coding-agent/src/main.ts` and `src/core/agent-session-services.ts`.
- Session construction and prompt preprocessing: `packages/coding-agent/src/core/sdk.ts` and `src/core/agent-session.ts`.
- Low-level loop and tool execution: `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.
- Provider dispatch: `packages/ai/src/compat.ts`, `src/models.ts`, and `src/types.ts`.

- Extension contract and loading: `packages/coding-agent/src/core/extensions/` plus `packages/coding-agent/docs/extensions.md`.

The repository evolves. If a symbol named in this book moves, use `rg "symbolName" packages` from the repository root, then update your mental model from the current implementation rather than preserving an old path.

2 TypeScript used by Pi

Pi is written as native ECMAScript modules in TypeScript. The root `package.json` sets `"type": "module"`, and source imports include a `.ts` suffix. The root `tsconfig.json` maps package names to their source entry points during repository type checking; published packages instead expose compiled `dist/` files through their package manifests.

You do not need to learn every TypeScript feature before contributing. The small set below covers the syntax that appears repeatedly in the runtime and extension API.

2.1 Modules and imports

An ES module exports named values or one default value. Import values normally; prefix an import with `type` when it exists only to describe data at compile time. A type-only import disappears from JavaScript output.

```
import { Type } from "@earendil-works/pi-ai";
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  // Type is a runtime value; ExtensionAPI is erased after type checking.
  void pi;
  void Type;
}
```

This is similar to Python importing a protocol only for static checking, except TypeScript makes the runtime/type distinction explicit. Pi follows this style throughout; for example, `packages/coding-agent/examples/extensions/hello.ts` imports `ExtensionAPI` as a type.

When reading repository source, relative imports normally include `.ts`:

```
import { Agent } from "../agent.ts";
```

Keep that convention in source packages. Do not introduce dynamic imports just to avoid a type cycle; Pi's project rules require top-level imports.

2.2 Types describe values; they do not validate values

An **interface** describes the fields a value must have. A **type** can name an interface-like shape, a union, or a computed type. Neither emits a runtime validator.

```
interface RunOptions {
  timeoutMs?: number;
  signal?: AbortSignal;
}

type RunState = "idle" | "streaming";
```

? means optional. The equivalent Python intuition is a dataclass field that may be omitted, but TypeScript callers can still supply arbitrary JSON unless you validate it at a boundary. Pi validates LLM tool arguments using schemas, not just TypeScript annotations.

2.3 Narrowing unions with a discriminant

Pi models events and content as discriminated unions: every variant has a literal tag such as **type** or **role**. An **if** or **switch** on that tag narrows the remaining fields safely.

```
type Content =
  | { type: "text"; text: string }
  | { type: "image"; mediaType: string; data: string };

function describe(content: Content): string {
  switch (content.type) {
    case "text":
      return content.text;
    case "image":
      return `image: ${content.mediaType}`;
  }
}
```

In Pi, the important real examples are **AgentEvent** in `packages/agent/src/types.ts`, assistant content blocks in `packages/ai/src/types.ts`, and session entries in `packages/coding-agent/src/core/session-manager.ts`. The low-level agent loop switches on streamed event types such as `"text_delta"`, `"toolcall_end"`, and `"done"` in `packages/agent/src/agent-loop.ts`.

This pattern is safer than checking for a field with **in**: the tag documents the protocol and enables exhaustiveness checking when a new variant is added.

2.4 Generics: preserve a relationship between inputs and outputs

Generic parameters are placeholders supplied by callers. Read `T` as “some type chosen later,” not as a runtime value. Pi uses generics heavily to keep a tool’s schema, parsed parameters, and implementation aligned.

```
function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}

const result = first(["a", "b"]); // string | undefined
```

The `readonly` modifier says the function promises not to mutate the array. The result includes `undefined` because TypeScript does not assume an array is non-empty. Handle it before using a string method:

```
if (result !== undefined) {
  result.toUpperCase();
}
```

`Static<typeof schema>` is a common Pi-specific generic pattern. `TypeBox` creates a runtime schema and derives a matching static TypeScript type from it. The `defineTool` helper used in the shipped `hello.ts` example infers the `params` type from `parameters`, so the `execute` function sees `params.name` as a string.

2.5 unknown is the honest type for untrusted data

Use `unknown` for data whose shape has not been proven: JSON, extension data, or a caught error. You must narrow it before using it. Avoid `any`; it turns off checking and can conceal a bad provider payload or tool result.

```
function messageOf(value: unknown): string {
  return value instanceof Error ? value.message : String(value);
}
```

This exact defensive shape occurs in many Pi error paths. It matters most at boundaries: provider responses, parsed configuration, session migration, and extension code loaded with `jiti`.

2.6 Async code and cancellation

An `async` function returns a `Promise<T>`, so its caller uses `await` to get the eventual `T`. Pi streams model output with asynchronous iterables:

```
for await (const event of response) {  
  // Consume a provider event as it arrives.  
}
```

`AbortSignal` is Pi's cancellation token. The agent supplies it to provider streams and tool execution; an extension should pass it into APIs that support it rather than inventing a second cancellation mechanism.

```
const response = await fetch(url, { signal });
```

The agent loop converts tool exceptions into error results, and its stream contract represents ordinary provider failures as final stream messages. That is why error handling belongs at the boundary rather than in a broad, unstructured `try/catch` around the whole application.

2.7 Objects, spreads, and immutable snapshots

Pi frequently makes a shallow copy before changing an object:

```
const next = { ...previous, thinkingLevel: "high" };  
const copiedMessages = messages.slice();
```

The spread expression copies own properties; nested arrays and objects remain shared. This is not deep immutability. In the agent package, snapshots of message and tool arrays protect loop state from accidental array mutation; in the extension API, some event payloads are intentionally mutable. Follow the contract for the specific event rather than assuming every object is frozen.

2.8 Functions are values and callbacks define lifecycle seams

Handlers passed to `pi.on`, tool `execute`, renderer functions, and queue callbacks are ordinary functions. The API calls them later. Arrow functions capture surrounding variables; class methods use `this` and must keep their receiver context.

```
pi.on("session_start", async (_event, ctx) => {  
  ctx.ui.notify("Ready", "info");  
});
```

The underscore is a naming convention for an intentionally unused parameter. It does not change runtime behavior.

2.9 Classes and public fields

The TUI and `Agent` use classes because they own long-lived mutable state, methods, subscriptions, and cleanup. You will see explicit fields and constructor assignment rather than TypeScript parameter properties. That is intentional: the repository uses TypeScript syntax that Node can erase without a JavaScript emit transform.

```
class Counter {
  private value: number;

  constructor(initial: number) {
    this.value = initial;
  }

  increment(): number {
    this.value += 1;
    return this.value;
  }
}
```

`private` is a TypeScript compile-time access restriction. It is not a security boundary. Prefer a small public API over reaching into a coding-agent class's private state.

2.10 TypeBox: runtime schemas for tool inputs

The model produces JSON-like arguments, so Pi needs a runtime schema as well as a static type. `typebox` supplies schemas such as `Type.Object` and `Type.String`. Pi re-exports `Type` from `@earendil-works/pi-ai`, and validates tool calls in `packages/agent/src/agent-loop.ts` before execution.

```
import { Type } from "@earendil-works/pi-ai";

const parameters = Type.Object({
  name: Type.String({ description: "Name to greet" }),
});
```

For string choices used in a tool exposed to Google-compatible APIs, use `StringEnum` from `@earendil-works/pi-ai`; the extension documentation records that `Type.Union` plus `Type.Literal` is not compatible with Google's API.

2.11 A compact reading checklist

When TypeScript looks dense, answer these questions in order:

1. What is the runtime value, and what is type-only?
2. Which tag (`type`, `role`, or another literal) narrows this union?

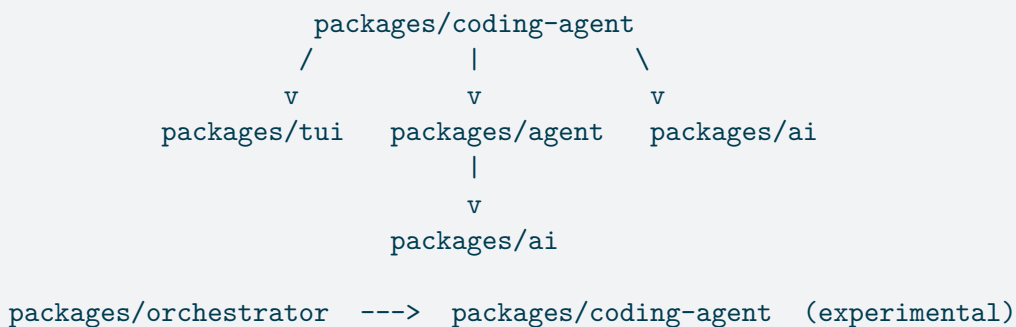
3. Which values may be missing (`?` or `| undefined`)?
4. Is the data external or model-produced and therefore runtime-validated?
5. Is this callback invoked now, or saved for a future lifecycle event?
6. Does the function receive an **AbortSignal** that must be forwarded?

That checklist is enough to read the following architecture and lifecycle chapters without treating types as magic.

3 Architecture: packages, ownership, and entry points

Pi deliberately separates generic agent machinery from the coding-agent application. A useful rule is: move a feature down only when it is genuinely useful outside the coding agent. The coding-agent package owns command-line behavior, local filesystem tools, project resources, session UX, and the extension API; the `agent` package should not learn those policies.

3.1 Dependency direction



The diagram shows the normal architectural direction, not every development tool dependency. `coding-agent` integrates all lower layers. `agent` depends on the AI message/model contracts. `tui` is independent of the agent and provider layers, allowing it to be reused by other terminal applications.

3.2 Repository-level conventions

The root `package.json` defines npm workspaces for `packages/*` and selected extension examples. All packages are ESM. The root `tsconfig.json` maps public package names back to source files for whole-repository checking. Each package has its own `package.json` with public exports and build script.

When developing inside this checkout, import the public package name if you are crossing a package boundary. Use a relative `.ts` import only within a package. This preserves the boundary that published consumers see.

The normal quality gate after code changes is `npm run check`. Per repository rules, do not run `npm run build` or the full test suite merely as a habit; use the requested or relevant targeted validation. This guide adds no runtime code, so its separate validation is Quarto rendering.

3.3 Component walkthrough

3.3.1 packages/tui: terminal rendering primitives

`packages/tui/src/index.ts` is the public barrel. It exports the TUI class, the `Component` interface, containers, text/editor/list components, terminal abstractions, key parsing, keybindings, autocomplete, and ANSI-aware text utilities.

`packages/tui/src/tui.ts` owns the component tree, focus, layout, overlays, input routing, and differential rendering. `src/terminal.ts` provides the process-terminal boundary. The components under `src/components/` render specific UI pieces; `src/keybindings.ts`, `src/keys.ts`, and the editor components handle keyboard semantics. This package does not know what a model, tool call, session, or extension is.

Use this layer when a change is a general terminal interaction primitive. For a Pi-specific chat row, command, or tool renderer, start in coding-agent instead.

3.3.2 packages/ai: a provider-neutral streaming contract

The small public entry point `packages/ai/src/index.ts` exports core types, models, authentication utilities, event-stream helpers, `TypeBox`, and selected API option types. It intentionally avoids eagerly importing provider catalogs and provider factories. Provider-specific modules live under `src/providers/` and streaming implementations under `src/api/`.

The core concepts are defined in `src/types.ts`:

- A `Model` carries provider/API metadata and capabilities.
- A `Context` combines the system prompt, normalized messages, and tool definitions.
- An `AssistantMessageEventStream` yields provider-independent start, text, thinking, tool-call, completion, and error events.

`src/compat.ts` supplies the compatibility `streamSimple()` dispatch used on the coding-agent path. It chooses a built-in model registry or resolves an API provider from `model.api`, then passes a `Context` plus simple stream options to that provider. `src/models.ts` exposes a `Models` implementation that owns provider registration and routes model requests. Individual files such as `src/api/openai-responses.ts` and `src/api/anthropic-messages.ts` serialize the normalized context into a provider request and translate that provider's stream back into Pi events.

Provider adapters are the right place for protocol-specific JSON, streaming parsing, or provider quirks. Do not put a UI decision or a built-in filesystem tool in this package.

3.3.3 packages/agent: generic conversation and tool loop

`packages/agent/src/types.ts` declares the generic agent contract: agent messages, tools, events, loop configuration, hooks, queue modes, and tool execution mode. `src/agent.ts` wraps that loop in the stateful `Agent` class. It owns current model, system prompt, tools, messages, active runs, abort controllers, and steering/follow-up queues.

`src/agent-loop.ts` is the important execution core. It:

1. Emits agent/turn/message events.
2. Lets a caller transform agent messages, then converts them to LLM messages.
3. Builds a provider `Context` and consumes an async response stream.
4. Validates model-produced tool arguments.
5. Runs tools sequentially or with sequential preflight plus parallel execution.
6. Appends tool results and repeats until there are no tool calls or queued messages.

The generic loop deliberately receives callbacks such as `convertToLlm`, `transformContext`, `beforeToolCall`, and `afterToolCall`. It therefore has no dependency on `SessionManager`, local disk tools, or the interactive TUI.

3.3.4 packages/coding-agent: the Pi application

This is where the generic pieces become the `pi` command. Its package manifest defines the `pi` binary as `dist/cli.js`. In source, `src/cli.ts` sets process-level behavior and calls `main(process.argv.slice(2))` from `src/main.ts`.

`src/main.ts` parses command-line arguments, determines interactive/RPC/print mode, resolves trust and settings, creates the runtime, selects the model, and starts `InteractiveMode`, RPC mode, or print mode. It is composition and application policy, not the agent loop.

The major coding-agent subsystems are:

Subsystem	Responsibility	Important files
Session composition	Create settings, auth, model registry, resources, agent session	<code>core/sdk.ts</code> , <code>core/agent-session-services.ts</code>
Runtime replacement	Own a session plus cwd-bound services across new/resume/fork	<code>core/agent-session-runtime.ts</code>
Session policy	Prompt preprocessing, resource/system prompt assembly, retries, compaction, extension event bridge	<code>core/agent-session.ts</code>
Durable transcript	JSONL session entries, tree branches, context reconstruction	<code>core/session-manager.ts</code> , <code>core/messages.ts</code>
Resources	Load extensions, skills, prompts, themes, and context files	<code>core/resource-loader.ts</code> , <code>core/extensions/loader.ts</code>
Configuration	Global/project settings and defaults	<code>core/settings-manager.ts</code> , <code>core/defaults.ts</code>
Model/auth resolution	Registered models, dynamic providers, credential lookup	<code>core/model-registry.ts</code> , <code>core/auth-storage.ts</code>
Built-in tools	Read, bash, edit, write, search, file mutation coordination	<code>core/tools/</code> , <code>utils/tools-manager.ts</code>
Interactive app	Chat transcript, editor, dialogs, commands, renderer adapters	<code>modes/interactive/interactive-mode.ts</code> , <code>modes/interactive/components/</code>
Extensions	Types, discovery/loading, event dispatch, context wrappers	<code>core/extensions/types.ts</code> , <code>loader.ts</code> , <code>runner.ts</code> , <code>wrapper.ts</code>

The table is an orientation map, not an exhaustive API list. Public coding-agent exports come from `src/index.ts`; extension authors should import from the package root rather than internal paths unless an API is explicitly documented for that purpose.

3.3.5 packages/orchestrator: experimental process orchestration

`packages/orchestrator` depends on `coding-agent` and exports configuration, handlers, IPC client/server/protocol, process supervision, storage, and serve functions from `src/index.ts`. Its manifest describes it as experimental. Treat it as a separate surface for multi-process orchestration, not as a hidden step of normal prompt processing.

3.4 Constructing a running coding-agent session

The startup path is intentionally split so the runtime can be rebuilt when a session changes its working directory.

```
main.ts
-> createAgentSessionRuntime(...)
  -> createAgentSessionServices(...)
    SettingsManager + AuthStorage + ModelRegistry + DefaultResourceLoader
  -> createAgentSessionFromServices(...)
    -> createAgentSession(...) in core/sdk.ts
      -> Agent + AgentSession + built-in/custom tool registry
-> InteractiveMode | runRpcMode | runPrintMode
```

`createAgentSessionServices()` reloads the resource loader and applies provider registrations that extensions made during their factories. `createAgentSession()` then resolves/restores a model and thinking level, creates an `Agent`, and wraps it in `AgentSession`. The `AgentSessionRuntime` class tears down the old extension/session runtime before applying a newly created one for operations such as session resume or fork.

3.5 Persistence and context are different things

Pi sessions are JSONL files managed by `SessionManager`. They are tree-shaped: entries have `id` and `parentId`, so a branch can exist in one file. The current branch is converted into LLM context by `buildContextEntries()` and related helpers in `core/session-manager.ts`.

Not every durable entry reaches the provider. A custom entry written with `pi.appendEntry()` persists extension state but does not participate in LLM context. A custom message sent with `pi.sendMessage()` does participate. This distinction is central to extension design: keep private UI/state metadata out of model context unless the model needs it.

Compaction is another context transformation, not deletion. Pi writes a summary entry and reconstructs a compact context from the summary and newer entries. The generic algorithms are exported by `packages/agent` under `src/harness/compaction/`; coding-agent coordinates the user-facing policy and session entries.

3.6 Where to make a change

Use the narrowest layer that truthfully owns the behavior:

Desired change	First place to investigate
Add a provider/API serializer	<code>packages/ai/src/api/</code> and <code>src/providers/</code>
Change generic tool scheduling or streaming semantics	<code>packages/agent/src/agent-loop.ts</code>
Add keyboard/editor/component capability reusable by TUIs	<code>packages/tui/src/</code>
Change Pi commands, session behavior, built-in tool policy, settings	<code>packages/coding-agent/src/</code>
Add user/team behavior without forking Pi	Extension, skill, theme, prompt, settings, or package

The next chapter traces this layering one prompt at a time.

4 Prompt lifecycle: from Enter to the next render

This chapter traces the normal interactive path. It is source-grounded in `packages/coding-agent/src/modes/interactive/interactive-mode.ts`, `packages/coding-agent/src/core/agent-session.ts`, `packages/agent/src/agent.ts`, `packages/agent/src/agent-loop.ts`, and the `packages/ai` dispatch layer.

The exact call graph has guards for commands, compaction, retries, queued messages, and other modes. The numbered path below is verified behavior; its diagrams omit helper calls that do not alter the data/control boundary being explained.

4.1 The complete path

Editor Enter

```
-> InteractiveMode.getUserInput()
-> AgentSession.prompt(text)
  -> extension command / input preprocessing / prompt expansion
  -> user + extension custom messages
  -> Agent.prompt(messages)
    -> agentLoop
      -> transform context -> convert messages -> provider stream
      -> assistant stream events -> tool validation/execution
      -> tool-result messages -> repeat or finish
    -> session persistence + extension events + transcript rendering
```

4.2 Stage 1: Terminal input enters the TUI

`InteractiveMode` owns the coding-agent's interactive screen. Its editor submission handler recognizes built-in slash commands and `!/!!` user-shell commands before normal agent prompting. A normal idle submission is queued or resolved through `getUserInput()`. The `run()` loop awaits that method, then calls `await this.session.prompt(userInput)`.

When `Pi` is already streaming, regular Enter calls `session.prompt(text, { streamingBehavior: "steer" })`; Alt+Enter uses "followUp". That is not a second concurrent agent run. It queues a message for a defined loop boundary. During compaction, non-command input is held by interactive mode until the compaction flow completes.

The initial prompt supplied by command-line arguments travels through the same `session.prompt()` API. Print and RPC modes do not instantiate `InteractiveMode`, but they converge on `AgentSession` rather than reimplementing the provider path.

4.3 Stage 2: Commands, keybindings, and input interception

There are two distinct command systems:

- Built-in interactive commands such as `/model`, `/settings`, `/new`, and `/compact` are handled in `InteractiveMode` before a normal prompt reaches the session.
- Extension commands registered with `pi.registerCommand()` are detected first inside `AgentSession.prompt()`. They execute immediately, including while an agent is streaming, and do not enter the model context unless their handler explicitly sends a message.

For ordinary text, `AgentSession.prompt()` asks the extension runner to emit an `input` event. The event sees raw text after extension-command lookup and before skill/template expansion. An extension can continue, transform, or handle it. If it continues, Pi expands `/skill:name ...` and prompt templates, then continues with the expanded text.

This order matters for custom syntax. An extension that wants to intercept a skill invocation must use `input`; one that wants a durable custom command should use `registerCommand`, not parse slash text manually.

4.4 Stage 3: Agent-session orchestration constructs a turn

Before starting a fresh run, `AgentSession.prompt()` rejects missing model or credential cases, checks pre-prompt compaction, builds a user `AgentMessage`, adds pending `nextTurn` custom messages, and emits `before_agent_start`. Handlers can add custom messages and chain a system-prompt override. The session then calls its private `_runAgentPrompt(messages)` method.

`_runAgentPrompt()` calls `Agent.prompt()` and may later call `Agent.continue()` for automatic retry, automatic compaction retry, or messages queued by an `agent_end` handler. It finally clears the per-run prompt override, flushes pending user-bash messages, and emits `agent_settled`.

The `Agent` class in `packages/agent/src/agent.ts` owns the active abort controller, streaming flags, accumulated messages, active tools, and queues. `Agent.prompt()` captures a context snapshot and drives `runAgentLoop()`.

4.5 Stage 4: Context and system prompt are built at two boundaries

The coding agent first builds a base system prompt in `AgentSession._rebuildSystemPrompt()`. It combines the working directory, loaded `AGENTS.md/context` files, loaded skills, custom and appended system prompts, and selected tools plus their snippets and guidelines.

`buildSystemPrompt()` in `core/system-prompt.ts` converts that structured data to text. Changing active tools rebuilds this base prompt, so tool prompt metadata can affect the next model turn.

Immediately before *each* model request, the low-level loop has another boundary. `streamAssistantResponse()` in `agent-loop.ts` calls the optional `transformContext` callback on `AgentMessage[]`, then calls `convertToLlm()` to produce provider-compatible `Message[]`. Coding-agent supplies these callbacks when creating `Agent` in `core/sdk.ts`:

- `transformContext` dispatches the extension `context` event.

- `convertToLlm` filters/normalizes coding-agent-specific message types and applies the image-blocking setting as a defense in depth.

The resulting `Context` has exactly `systemPrompt`, `messages`, and `tools`. This separation explains why a session can persist rich UI-only entries while only a selected representation reaches the LLM.

4.6 Stage 5: Model and provider selection

At startup or session restoration, `createAgentSession()` in `core/sdk.ts` tries the model recorded in a nonempty session, then configured defaults, then provider defaults through `findInitialModel()`. It clamps the thinking level to the selected model's capabilities.

The coding agent's `ModelRegistry` owns the model catalog, configured provider overrides, credential lookup, OAuth state, and request headers. The stream function installed by `core/sdk.ts` calls `modelRegistry.getApiKeyAndHeaders(model)`, merges provider-attribution and request headers, then emits the extension `before_provider_headers` hook.

`packages/ai/src/compat.ts` receives the chosen model and its `api` identifier. `streamSimple()` either delegates to built-in model support or resolves the registered API provider and calls that provider's `streamSimple`. Provider adapters in `packages/ai/src/api/` serialize the normalized context and return a provider-independent assistant event stream.

Extensions can participate at three precise points:

- `before_provider_headers` mutates assembled request headers.
- `before_provider_request` receives and may replace the provider-specific payload after serialization.
- `after_provider_response` observes HTTP status/headers before stream consumption when the adapter exposes them.

Those hooks are not substitutes for a custom provider. Use `pi.registerProvider()` when you must declare a model catalog, endpoint, auth, or streaming implementation.

4.7 Stage 6: Streaming response becomes agent events and TUI updates

`streamAssistantResponse()` consumes the `AssistantMessageEventStream` with `for await`. It creates a partial assistant message on `start`, replaces that partial message as text/thinking/tool-call deltas arrive, and emits `message_start`, `message_update`, and `message_end` events.

`Agent.processEvents()` reduces those events into state and then awaits all subscribers. `AgentSession` bridges generic events to persistence, extensions, and application behavior. `InteractiveMode` updates the visible transcript, current streaming row, tool rows, footer, and render requests. The TUI package performs the actual terminal rendering.

The extension lifecycle maps directly onto this path: `agent_start`, `turn_start`, `message_start`, many `message_update` events, `message_end`, then later `turn_end` and `agent_end`.

4.8 Stage 7: Tool calls are parsed, validated, gated, and executed

After a final assistant message, `agent-loop.ts` filters content blocks whose `type` is `"toolCall"`. A message stopped for `"length"` fails all its tool calls instead of executing possibly truncated JSON arguments.

For each normal tool call, the loop:

1. Finds an active tool by name from the current context.
2. Runs its optional `prepareArguments` compatibility transform.
3. Calls `validateToolArguments()` against the `TypeBox` schema.
4. Emits `tool_execution_start`.
5. Invokes `beforeToolCall`, which coding-agent wires to extension `tool_call`. A handler may mutate validated input or return `{ block: true }`.
6. Calls the tool's `execute(id, args, signal, onUpdate)` function.
7. Converts incremental `onUpdate` values into `tool_execution_update` events.
8. Calls `afterToolCall`, wired to extension `tool_result`, and allows its partial result patch.
9. Emits `tool_execution_end` and saves a `toolResult` message.

The configured default is parallel execution: preflight happens in assistant source order, successful tool executions may run concurrently, and final `toolResult` messages are emitted in assistant source order. Start/update/end event ordering differs in parallel mode; see the extension documentation before using sibling tool results as state.

Built-in tools are supplied by coding-agent. They are ordinary tools at this boundary, which is why an extension can add a tool or override a built-in one. Custom filesystem-mutating tools must participate in `withFileMutationQueue()` to avoid lost writes when sibling tools run in parallel.

4.9 Stage 8: Results create the next agent action

Each final tool result is appended to the in-memory agent context and becomes the model-facing material for the next turn. If at least one tool call was executed and the batch was not explicitly terminating, the loop starts another assistant response. If the model instead returns ordinary final text, the loop checks steering messages first and follow-up messages when it would otherwise stop.

```
user/custom context -> model -> assistant tool calls
                        |
                        v
                    validated tool results
                        |
                        +----> model again
```

A tool can return `terminate: true` to hint that the next model call should be skipped, but the batch terminates only when every finalized tool result sets that flag. A tool signals failure by throwing; merely returning a field named `isError` does not mark it failed.

4.10 Stage 9: Persistence and rendering settle the turn

As message events complete, `AgentSession` records them through its `SessionManager`. The session is a JSONL tree, so the persisted form can later rebuild a branch and compact context. Model and thinking-level changes are also session entries. Extension state added with `appendEntry()` is persisted as a custom entry but remains outside LLM context.

`InteractiveMode.handleEvent()` responds to the same event stream. Assistant updates repaint the streaming message; tool lifecycle events manage pending tool rows; finalized messages become permanent transcript components. Renderer registration lets an extension replace the display of its custom messages, entries, or tool calls/results without changing how the model loop works.

4.11 Stage 10: Errors, cancellation, retries, compaction, and shutdown

There are several deliberately different failure paths:

Situation	What happens
Provider/model run throws	<code>Agent.runWithLifecycle()</code> turns it into a final assistant message with <code>stopReason: "error"</code> or <code>"aborted"</code> , then emits normal terminal events.
Provider stream reports error/aborted	<code>agent-loop.ts</code> ends the turn without executing tools.
Tool schema invalid, unknown tool, hook blocks, or execute throws	The loop creates an error tool result and gives that result to the model.
User presses Esc / abort is requested	The active <code>AbortController</code> is aborted; abort-aware provider/tool/extension work should receive the signal.
Retryable assistant error	<code>AgentSession</code> may prepare an automatic retry after the low-level run ends.
Context pressure	<code>AgentSession</code> may compact, write a summary entry, rebuild context, and continue/retry.
Session replacement or process exit	The extension runtime receives <code>session_shutdown</code> ; session-bound resources must be cleaned up.

`agent_end` means the current low-level loop stopped. `agent_settled` is later: it means Pi has no automatic retry, compaction retry, or queued continuation left. Status integrations and cleanup that require true idleness should use `agent_settled` or `ctx.waitForIdle()` in a command context.

4.12 Trace this yourself

For a live source trace, start at these symbols in order:

1. `InteractiveMode.run()` and `getUserInput()`.
2. `AgentSession.prompt()` and `_runAgentPrompt()`.

3. `Agent.prompt()`, `createLoopConfig()`, and `processEvents()`.
4. `runAgentLoop()`, `streamAssistantResponse()`, and `executeToolCalls()`.
5. `streamSimple()` in `packages/ai/src/compat.ts`.
6. The `streamSimple` function in the selected provider API adapter.

Use the extension event timeline in `packages/coding-agent/docs/extensions.md` as a second view of the same lifecycle. It documents what extension authors can rely on; the source symbols above show which layer makes each guarantee.

5 Extensions: change Pi without forking it

An extension is a TypeScript or JavaScript module that Pi loads at runtime with `jiti`. Its default export is a factory receiving `ExtensionAPI`. The factory registers hooks, tools, commands, shortcuts, flags, renderers, and providers; Pi later invokes those registrations at defined lifecycle boundaries.

Extensions execute with the permissions of the Pi process. They are trusted code, not a sandbox. Review a package before installing it and treat an extension's filesystem, process, network, and credential access as you would a normal local program.

5.1 Choose the right customization mechanism

Before writing code, use the smallest mechanism that owns the behavior:

Need	Prefer
Persist a model instruction or reusable task procedure	<code>AGENTS.md</code> , a skill, or a prompt template
Change colors and visual style	Theme
Change model/provider selection or static behavior	Settings or a custom provider
Add a command, tool, lifecycle gate, state, custom UI, or request hook	Extension
Reuse Pi from another Node program	Coding-agent SDK (<code>createAgentSession</code>)
Change generic loop/provider/TUI behavior for every consumer	Core contribution in the owning package

An extension is powerful but should not become an unreviewed fork of the interactive application. Favor public `ExtensionAPI` methods over importing coding-agent internals.

5.2 Discovery, trust, and loading

Pi automatically discovers a single-file extension or directory `index.ts` in these places:

Scope	Files discovered
Global	<code>~/.pi/agent/extensions/*.ts</code> , <code>~/.pi/agent/extensions/*/index.ts</code>
Project	<code>.pi/extensions/*.ts</code> , <code>.pi/extensions/*/index.ts</code>

Project-local resources are loaded only after the project is trusted. You can also add local paths in `settings.json`, or try a path for one invocation with `pi -e ./path/to/extension.ts`. The latter is ideal for a quick test, but `/reload` hot reload is intended for auto-discovered locations.

At startup, the resource loader loads extension factories before session start. An async factory is awaited before startup continues. That makes it suitable for one-time initialization such as discovering models and calling `pi.registerProvider()`. Do not start timers, watchers, processes, or sockets in the factory: commands such as `--list-models` can load factories without starting a session. Start long-lived session resources in `session_start` and close them idempotently in `session_shutdown`.

5.3 The lifecycle you can rely on

```
factory runs
-> project_trust (eligible global/CLI extensions only)
-> session_start
-> resources_discover

prompt
-> extension commands, then input
-> before_agent_start -> agent_start
-> for each model/tool turn:
    turn_start -> context -> provider hooks -> streamed messages
    -> tool execution hooks -> turn_end
-> agent_end -> agent_settled

reload/switch/fork/quit -> session_shutdown
```

The timeline is condensed. The authoritative event names, return values, and parallel-tool ordering are in `packages/coding-agent/docs/extensions.md` and the type definitions in `packages/coding-agent/src/core/extensions/types.ts`. Use the types for autocomplete and the documentation for behavioral contracts.

5.3.1 Event groups and common use

Event group	Representative events	Use it for
Trust/resources	<code>project_trust</code> , <code>resources_discover</code>	Decide project trust or contribute paths for skills/prompts/themes
Session	<code>session_start</code> , <code>session_shutdown</code> , <code>session_before_switch</code> , <code>session_before_compact</code>	Initialize/clean resources, persist/rebuild state, gate a session operation
Input/agent	<code>input</code> , <code>before_agent_start</code> , <code>agent_start</code> , <code>agent_end</code> , <code>agent_settled</code>	Transform input, add prompt context, track the true idle state

Event group	Representative events	Use it for
Turn/messages	<code>turn_start</code> , <code>context</code> , <code>message_update</code> , <code>turn_end</code>	Adjust context or render/measure streaming activity
Provider	<code>before_provider_headers</code> , <code>before_provider_request</code> , <code>after_provider_response</code>	Tracing, a request rewrite, response metadata
Tool	<code>tool_execution_start</code> , <code>tool_call</code> , <code>tool_execution_update</code> , <code>tool_result</code> , <code>tool_execution_end</code>	Observe progress, block/mutate calls, patch results
Model	<code>model_select</code> , <code>thinking_level_select</code>	Update model-aware UI/state
User shell	<code>user_bash</code>	Route ! and !! through a different execution backend

`tool_call` is a policy enforcement point: it runs after schema validation and can block a tool. Its input is intentionally mutable, so a mutation affects execution and later hook handlers see the changed input. Pi does not validate again after that mutation. Keep any mutation narrow and validate your own invariants before relying on it.

`context` receives a deep copy of messages that an extension may replace. It is a safer place to prune/inject LLM context than modifying session storage. `before_provider_request` is lower-level still: it sees the provider-specific payload after serialization, so changes there are not reflected by `ctx.getSystemPrompt()`.

5.4 A complete first extension

This extension adds a model-callable `hello` tool, a `/hello-pi` command, and a startup notification. It intentionally has no dependencies beyond Pi's bundled extension APIs.

1. Create the file in an auto-discovered global location:

```
mkdir -p ~/.pi/agent/extensions
```

2. Save this as `~/.pi/agent/extensions/hello-pi.ts`:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet a person by name",
  parameters: Type.Object({
```

```

    name: Type.String({ description: "The person to greet" }),
  }),
  async execute(_toolCallId, params, _signal, _onUpdate, _ctx) {
    return {
      content: [{ type: "text", text: `Hello, ${params.name}!` }],
      details: { greeted: params.name },
    };
  },
});

export default function register(pi: ExtensionAPI) {
  pi.registerTool(helloTool);

  pi.registerCommand("hello-pi", {
    description: "Show a greeting notification",
    handler: async (args, ctx) => {
      ctx.ui.notify(`Hello, ${args || "world"}!`, "info");
    },
  });

  pi.on("session_start", async (_event, ctx) => {
    ctx.ui.notify("hello-pi extension is ready", "info");
  });
}

```

The tool definition uses `defineTool`, as does the repository's verified `packages/coding-agent/examples/extensions/hello.ts`. `Type.Object` is a runtime schema. `defineTool` derives the static type of `params`, and the agent loop validates model-supplied arguments against that schema before calling `execute`.

3. Start Pi in any project. The startup notification confirms discovery. Run `/hello-pi Ada` to exercise the command. Ask Pi to greet Ada with the `hello` tool, or inspect its active tool list, to exercise the model-facing tool.
4. Edit the file and run `/reload` in Pi. The reload path emits `session_shutdown`, reloads resources, then emits `session_start` with `reason: "reload"`. Treat code after `await ctx.reload()` as stale; the documented safe pattern is `await ctx.reload(); return`.

For a one-off test outside discovery, use `pi -e ~/.pi/agent/extensions/hello-pi.ts`. The global file is preferable for the reload workflow.

5.5 What ExtensionAPI and context provide

`ExtensionAPI` is the registration/control surface. Its important methods are:

API	Purpose
<code>pi.on()</code>	Subscribe to a typed lifecycle event

API	Purpose
<code>pi.registerTool()</code>	Add or override a model-callable tool
<code>pi.registerCommand()</code>	Add <code>/name</code> command with optional argument completion
<code>pi.registerShortcut()</code> / <code>pi.registerFlag()</code>	Add a keybinding action or CLI option
<code>pi.sendMessage()</code> / <code>pi.sendUserMessage()</code>	Put a custom or user message into agent flow
<code>pi.appendEntry()</code>	Persist extension-only session data outside LLM context
<code>pi.registerMessageRenderer()</code> / <code>pi.registerEntryRenderer()</code>	Render custom message/entry types in the transcript
<code>pi.getActiveTools()</code> / <code>pi.setActiveTools()</code>	Inspect or change the currently exposed tool set
<code>pi.setModel()</code> / <code>pi.setThinkingLevel()</code>	Change live session model settings
<code>pi.registerProvider()</code> / <code>pi.unregisterProvider()</code>	Register an endpoint/model provider at runtime
<code>pi.exec()</code>	Run a process with Pi's helper interface

Handlers receive a context. The high-value fields are `ctx.cwd`, `ctx.mode`, `ctx.hasUI`, `ctx.sessionManager`, `ctx.modelRegistry`, `ctx.model`, `ctx.signal`, and `ctx.ui`. Use `ctx.signal` for cancelable nested work. Use `ctx.mode === "tui"` before requiring terminal-only UI behavior and `ctx.hasUI` before prompting the user.

`ExtensionCommandContext` adds session-changing methods such as `newSession`, `fork`, `navigateTree`, `switchSession`, `waitForIdle`, and `reload`. They are available only in command handlers because changing a session from an ordinary event handler can deadlock or leave stale state.

5.6 Tools: make the model contract explicit

A custom tool needs a unique `name`, human-facing `label`, model-facing `description`, `TypeBox` `parameters`, and `execute`. The return `content` goes to the LLM; `details` is structured metadata for renderers and state recovery.

Add `promptSnippet` when the tool should have a one-line entry in Pi's default system prompt. Add `promptGuidelines` for model instructions while the tool is active. Each guideline must name its tool because Pi flattens all active tool guidelines into one section.

For robust tools:

- Throw from `execute` to report an error to the model.
- Forward `signal` into `fetch`, process helpers, and long computations.
- Use `onUpdate` for incremental result rendering.
- Truncate large output with the exported truncation helpers; built-in limits are 50 KB or 2,000 lines.
- Use `StringEnum` from `@earendil-works/pi-ai` for string enums exposed to Google-compatible APIs.
- Use `withFileMutationQueue(absolutePath, callback)` around the *entire* read-modify-write window for a file-mutating tool.

An extension may override a built-in tool by registering the same name. That is an advanced compatibility commitment: preserve the built-in result `details` shape expected by session and UI code. If the override omits a renderer, Pi can reuse the built-in renderer slot, but prompt metadata is not inherited.

5.7 Commands, UI, and rendering

Commands are best for direct user actions and configuration. If multiple extensions register the same name, Pi keeps both and gives them numeric invocation suffixes in load order. Do not assume a command name is globally unique after loading third-party packages.

`ctx.ui` supports notifications, select/confirm/input/editor dialogs, footer status, widgets, editor text, themes, custom autocomplete, tool expansion, and replacement editors. For a multi-key interaction, `ctx.ui.custom()` temporarily replaces the editor with a `@earendil-works/pi-tui` component. In RPC and noninteractive modes, many TUI-specific operations are no-ops or return defaults; check mode first.

Use a message renderer for a custom message created with `pi.sendMessage()`; that content participates in LLM context. Use an entry renderer for a custom entry created with `pi.appendEntry()`; that is durable transcript/UI state but does not reach the LLM. This is the key display-versus-context choice.

5.8 State, sessions, and queues

In-memory variables vanish on reload, resume, fork, and restart. Reconstruct state in `session_start` by examining the active branch. For tool-backed state, store a recoverable snapshot in that tool result's `details`; it follows the conversation branch naturally. Use `appendEntry()` for state that should not enter model context.

`pi.sendMessage()` accepts delivery modes: `steer` inserts at the next steering boundary, `followUp` waits until the agent would finish, and `nextTurn` waits for the next user prompt. `sendUserMessage()` creates a real user message and requires a delivery mode while streaming. Do not start a second direct agent run from a hook; use these queues.

5.9 Distribute an extension as a Pi package

For a reusable package, declare explicit resources in `package.json`:

```
{
  "name": "@acme/pi-team-tools",
  "keywords": ["pi-package"],
  "peerDependencies": {
    "@earendil-works/pi-ai": "*",
    "@earendil-works/pi-agent-core": "*",
    "@earendil-works/pi-coding-agent": "*",
    "@earendil-works/pi-tui": "*"
  }
}
```

```

    "typebox": "*"
  },
  "pi": {
    "extensions": ["/extensions"],
    "skills": ["/skills"],
    "prompts": ["/prompts"],
    "themes": ["/themes"]
  }
}

```

Put third-party runtime packages in `dependencies`, not `devDependencies`: Pi package installation uses production installs by default. Core Pi packages are bundled by Pi, so list them as `peerDependencies` with the documented `"*"` range rather than bundling a competing copy. Package conventions also discover `extensions/`, `skills/`, `prompts/`, and `themes/` when a `pi` manifest is not present, but an explicit manifest is clearer and supports filtering.

Users can install a package from npm or git with `pi install`, or list it under the `packages` array in settings. Project-installed packages are trust-gated just like project-local extensions.

5.10 Debugging extensions

Start with the smallest observation point:

1. Confirm discovery with a `session_start` notification.
2. Use `pi -e path/to/file.ts` to isolate a one-run load failure.
3. Run `/reload` after edits in an auto-discovered location.
4. Use `ctx.ui.notify()` in TUI mode and concise diagnostic logging while developing.
5. Use the hidden `/debug` command for the rendered TUI lines and the last messages sent to the LLM; it writes `~/.pi/agent/pi-debug.log`.
6. Inspect `ctx.sessionManager` state rather than guessing what a branch or compaction contributed.

Errors in extension handlers are logged and Pi continues, except a `tool_call` handler error fails safe by blocking that tool call. Test policy hooks for both allow and deny paths. Test `session_shutdown` cleanup by reload, new session, resume, and exit rather than only by startup.

6 Customization patterns and contributor workflow

This chapter turns the architecture into decisions. It also calls out the mistakes that most often create an extension which appears to work once but fails on reload, in parallel tool mode, or outside the interactive terminal.

6.1 Configuration before code

Pi configuration is layered. The coding-agent package uses a global agent directory (normally `~/.pi/agent`) and project-local `.pi` resources. The `piConfig.configDir` package setting makes the config directory rebrandable, so extension code should import `CONFIG_DIR_NAME` when constructing a project path instead of hardcoding `.pi`.

Use settings for stable choices such as providers, model selection, themes, keybindings, trusted packages, and resource locations. Use an extension when a choice must react to events, require a tool, retain state, or manipulate UI.

Project configuration and resources are an execution boundary: untrusted projects do not load project-local extensions, packages, skills, or related dynamic resources. A global extension should call `ctx.isProjectTrusted()` before honoring project-local configuration of its own.

6.2 Pattern: a safety gate around a built-in tool

This is appropriate for a team policy that requires confirmation before a destructive command. It observes the existing `bash` tool instead of replacing all shell behavior.

```
import { isToolCallEventType, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("tool_call", async (event, ctx) => {
    if (!isToolCallEventType("bash", event)) return;
    if (!event.input.command.includes("rm -rf")) return;

    const allowed = await ctx.ui.confirm("Dangerous command", "Allow rm -rf?");
    if (!allowed) return { block: true, reason: "Blocked by user" };
  });
}
```

The call is already schema-validated when the hook runs. The `isToolCallEventType` guard provides a typed `event.input` for a built-in tool. This pattern must handle non-TUI modes deliberately:

`ctx.hasUI` can be false, so choose a safe default or a noninteractive policy rather than assuming a dialog exists.

Do not mutate a tool's input merely to implement policy when blocking is more honest. If you do mutate, remember no second schema validation occurs.

6.3 Pattern: inject focused context, not a second system

`before_agent_start` is the correct point to add per-prompt instructions. It can add a custom message and/or replace the chained system prompt. The custom message is stored in the session and reaches the LLM; a notification is not.

```
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("before_agent_start", async (event) => {
    if (!event.prompt.startsWith("review ")) return;
    return {
      message: {
        customType: "review-mode",
        content: "Prioritize correctness, regressions, and concise findings.",
        display: true,
      },
    };
  });
}
```

Multiple handlers see a chained system prompt. Prefer appending a narrow, conditional instruction over replacing the entire system prompt; replacement can accidentally discard skills, tools, context files, and user-provided instructions. For a message-level context transformation before *every* provider request, use the `context` event instead.

6.4 Pattern: state that survives restart and branching

For state derived from tool actions, put a serializable snapshot in each tool result's `details`, then reconstruct from `ctx.sessionManager.getBranch()` in `session_start`. A branch sees its own prior results, so it naturally restores the state associated with that branch.

```
tool result details -> durable session entry -> branch reload -> extension rebuilds state
```

For state that should be visible in the transcript but never sent to the model, use `pi.appendEntry("my-state", data)` and `pi.registerEntryRenderer("my-state", renderer)`. Do not use `sendMessage` for private bookkeeping; custom messages are model context.

6.5 Pattern: dynamic tools and active-tool policy

`pi.registerTool()` may be called after startup. Pi refreshes it immediately for the current session; it appears in `pi.getAllTools()` and can be enabled with `pi.setActiveTools()`. To avoid accidentally removing useful tools, start from the current selection:

```
const active = pi.getActiveTools();
pi.setActiveTools([...new Set([...active, "my_tool"])]);
```

Changing active tools affects the next turn and rebuilds the system prompt. `setActiveTools(["read", "bash"])` is a real policy change, not just a UI filter. Use it with a command/shortcut that clearly signals the new mode.

6.6 Pattern: provider customization

Use `pi.registerProvider()` to add a proxy, endpoint, model catalog, OAuth flow, or nonstandard stream implementation. Use a provider hook only for a small cross-cutting change such as tracing headers or payload inspection.

An async factory is the right location when the provider must query a local model service before registration. Pi waits for that factory, then applies queued registrations while it creates services. Registering from a later command takes effect immediately, but it cannot make the model available to earlier startup operations such as model listing.

Provider configuration has security impact. Environment-variable references for `apiKey` keep secrets out of source. Avoid logging headers, payloads, or `ctx.getSystemPromptOptions()` because they can contain credentials, context files, skills, and user instructions.

6.7 Pattern: custom TUI that still works outside the TUI

Use `ctx.ui.setStatus` for a small persistent footer line, `setWidget` for editor-adjacent information, and renderers for transcript content. Use `ctx.ui.custom()` only when you need an interactive component with its own keyboard behavior.

An extension may run in four modes: `"tui"`, `"rpc"`, `"json"`, and `"print"`. Dialogs and custom component behavior do not have equivalent semantics in every mode. Design a noninteractive fallback first, then add an interactive dialog:

```
if (!ctx.hasUI) {
  return { block: true, reason: "This action requires interactive approval" };
}
```

For custom editors and UI components, use injected keybinding managers and named keybinding IDs. Do not hardcode raw control-key tests; that bypasses user configuration.

6.8 Pattern: package a team customization

An extension package can bundle extensions, skills, prompts, and themes. Put runtime dependencies in `dependencies`; Pi package installation uses a production-style install. Declare Pi core libraries as documented peer dependencies. Share a versioned package with explicit `pi` manifest paths when the team needs repeatability, rather than copying a large extension into every repository.

Use a project package for project-specific behavior only after the team agrees to trust that source. Use a global package for behavior that is independent of the current project. The scope determines both discovery and the security boundary.

6.9 Common mistakes and the correct boundary

Mistake	Why it fails	Better approach
Start a file watcher in the extension factory	Factory may run for a non-session command	Start in <code>session_start</code> ; close in <code>session_shutdown</code>
Keep all extension state in a closure	Reload/resume/fork loses it	Rebuild from session entries or tool <code>details</code>
Call session replacement from a normal event	It can deadlock and leave stale contexts	Use a command and <code>ExtensionCommandContext</code>
Use <code>sendMessage</code> for private metadata	The model receives it	Use <code>appendEntry</code> and an entry renderer
Ignore <code>AbortSignal</code>	Esc cannot stop your nested work	Forward <code>ctx.signal</code> /tool signal
Mutate files in a custom tool without a queue	Parallel sibling calls can lose writes	Use <code>withFileMutationQueue</code> around read-modify-write
Override <code>read</code> or <code>bash</code> with a different result shape	Built-in UI/session behavior expects details	Preserve the documented built-in contract
Assume tool hooks observe sibling results	Parallel tools finish independently	Persist/coordinate after the batch or avoid the dependency
Assume <code>agent_end</code> means completely idle	Retry/compaction/follow-up may still run	Use <code>agent_settled</code> or command <code>waitForIdle()</code>
Test only <code>pi -e</code>	It does not prove discovery/reload behavior	Test from auto-discovery plus <code>/reload</code>

6.10 A practical verification ladder

Use evidence appropriate to what you changed:

1. Type-check the extension with the same TypeScript version/API contracts used by Pi, especially tool parameters and event return values.
2. Load it with `pi -e path/to/extension.ts` to isolate load errors.
3. Move it to an auto-discovered location and verify `session_start`.
4. Exercise the happy path and the relevant failure/deny/cancel path.
5. Run `/reload`, then repeat. Confirm no duplicate resource, stale closure, or leaked status remains.

6. For persistence, start a new process, resume/fork a session, and verify the intended branch state is restored.
7. For tools, test a malformed call/schema failure and a cancellation. For mutating tools, test concurrent sibling calls if applicable.
8. If you change repository code, run the repository-prescribed relevant check rather than a real-provider test. The coding-agent suite uses a faux provider under `packages/coding-agent/test/suite/harness.ts` for deterministic tests.

The repository ships many extension examples in `packages/coding-agent/examples/extensions/`. Read the smallest example that matches your need first: `hello.ts` for a tool, `permission-gate.ts` for a gate, `input-transform.ts` for input routing, `entry-renderer.ts` for transcript-only state, `dynamic-tools.ts` for runtime tool changes, and `subagent/` for a larger composition.

7 Sessions, branches, and durable context

Pi does not treat a chat as a flat array that is overwritten on every new turn. The coding agent persists an append-only JSONL session tree and rebuilds the current conversation from a selected leaf. That design supports resume, fork, branch navigation, compaction, labels, and extension state without requiring a database.

This chapter is grounded in `packages/coding-agent/src/core/session-manager.ts`, `packages/coding-agent/src/core/messages.ts`, `packages/coding-agent/src/core/agent-session-runtime.ts`, and `packages/coding-agent/docs/session-format.md`.

7.1 Why persistence is a tree

Suppose a user asks for an implementation, then navigates back to an earlier message and explores a different plan. A linear transcript either loses one path or duplicates a large prefix into another file. Pi instead stores entries with an `id` and `parentId`. One file can represent both paths.

```
header (not a tree node)
|
+-- user: "Investigate auth"
    |
    +-- assistant: plan A
        |
        +-- tool result: read auth.ts
            |
            +-- user: "Implement A"                <- old leaf
                |
                +-- branch summary: "A explored ..."
                    |
                    +-- user: "Try plan B instead" <- current leaf
```

The active branch is the chain from the current leaf to the root, read in chronological order. Other children are not provider context merely because they exist on disk. This is a critical distinction when debugging a surprising model response: inspect the active path rather than the whole session file.

7.2 Files, headers, and locations

By default, coding-agent sessions live under the agent directory, arranged by working directory. Each persisted session is a `.jsonl` file. The first line is a `session` header with metadata such as a UUID, timestamp, format version, and `cwd`; it has no `id`/`parentId` and is not part of the tree.

The session location can be overridden in settings or by CLI/environment configuration. The documented precedence is:

```
--session-dir > PI_CODING_AGENT_SESSION_DIR > settings.json sessionDir
```

Use `SessionManager.inMemory(cwd?)` when embedding Pi where persistence is not desired. An in-memory manager still has branches and context construction; it simply has no session-file path.

7.3 The entry vocabulary

Every non-header entry has a **type**, an **ID**, a **parent ID**, and a **timestamp**. The following table shows why each entry exists and whether it becomes model context through `buildSessionContext()`.

Entry type	Meaning	Becomes LLM context?
message	A standard AgentMessage : user, assistant, tool result, or supported coding-agent message	Yes, subject to branch/compaction selection
model_change	Selected provider/model at that point in history	Used to restore state, not a chat message
thinking_level_change	Selected thinking level	Used to restore state, not a chat message
compaction	Summary plus the first entry retained after summarization	Yes, as a compaction-summary message
branch_summary	Summary of the path left during tree navigation	Yes, as a branch-summary message
custom	Extension-specific durable data	No
custom_message	Extension message with display/details	Yes
label	Named marker attached to an entry	No
session_info	User-visible session name	No

This table yields a simple rule for extension authors: use `custom` for state that Pi or the user needs, and use `custom_message/pi.sendMessage()` only when the model should receive it.

7.4 Message shapes at the provider boundary

The `message` entry wraps an `AgentMessage`. The core roles are:

```
user          user text and optional images
assistant     streamed text, thinking, and/or toolCall blocks
toolResult    result for a specific assistant tool call
```

Coding-agent adds message kinds for local interaction and synthesized context, including user bash execution, custom messages, compaction summaries, and branch summaries. `convertToLlm` and the agent loop decide how these richer messages become the provider's normalized message types.

An assistant's content is a list of typed blocks. A tool call is one such block, containing an ID, name, and argument object. The corresponding `toolResult` refers to the call ID. Retaining that link is essential: a model provider expects a tool result to answer a particular tool call, not merely a similarly named tool.

7.5 SessionManager as the persistence boundary

The class exposes both static constructors/listing methods and instance methods. The most useful API groupings are:

Intent	Methods
Start/open	<code>create</code> , <code>open</code> , <code>continueRecent</code> , <code>inMemory</code> , <code>forkFrom</code>
Discover	<code>list</code> , <code>listAll</code>
Append transcript	<code>appendMessage</code> , <code>appendModelChange</code> , <code>appendThinkingLevelChange</code>
Append summaries/state	<code>appendCompaction</code> , <code>appendCustomEntry</code> , <code>appendCustomMessageEntry</code> , <code>appendLabelChange</code> , <code>appendSessionInfo</code>
Navigate tree	<code>getLeafId</code> , <code>getBranch</code> , <code>getTree</code> , <code>getChildren</code> , <code>branch</code> , <code>resetLeaf</code> , <code>branchWithSummary</code>
Build context	<code>buildContextEntries</code> , <code>buildSessionContext</code>
Inspect metadata	<code>getHeader</code> , <code>getSessionName</code> , <code>getCwd</code> , <code>getSessionFile</code> , <code>isPersisted</code>

The manager owns the JSONL representation. Do not have an extension write a second, unrelated copy of the session file: it will not participate in branch or compaction semantics and may become inconsistent with the transcript.

7.6 Building context from a selected leaf

`buildContextEntries()` is the intermediate view used by the interactive transcript and session context construction. At a high level it does this:

```
all persisted entries
  |
  v
walk parent links from active leaf to root
  |
  v
reverse into chronological active path
```

```
    |
    v
apply the latest effective compaction boundary
    |
    v
context entries for rendering and conversion
```

When there is no compaction, the active path is the selected context. With a compaction entry, Pi retains the compaction summary and only the entries from that entry’s `firstKeptEntryId` onward, plus later entries. It does not throw away the earlier raw records; they remain in the JSONL file for tree history, but they are intentionally not sent to the LLM.

`buildSessionContext()` goes a step further. It scans the full path to restore the current model and thinking level, then maps selected entries to context messages. Custom entries are omitted. This is why storing a private extension cache in `appendEntry()` does not inflate context length.

7.7 Compaction is a controlled replacement of old context

Context windows are finite. Pi’s automatic compaction condition is conceptually:

```
estimated context tokens > model context window - reserveTokens
```

The default reserve is 16,384 tokens and the default recent-history target is 20,000 tokens, both configurable under `compaction` in settings. The exact token calculation and cut-point behavior are implemented in the agent harness; coding-agent coordinates the session operation, extension events, UI, and retry behavior.

The standard flow is:

1. Locate a valid cut point near the recent-history token target.
2. Collect the older messages to summarize, keeping tool-result relationships valid.
3. Ask a model for a structured summary, including previous-summary context when necessary.
4. Append a compaction entry with summary, token count, and first kept entry.
5. Rebuild the session context from the summary plus retained recent entries.

Compaction is not the same as deleting history. Its purpose is to preserve an actionable description of old work while freeing provider context. An extension can intercept `session_before_compact` to cancel or provide a custom summary, and observes the final result through `session_compact`.

7.7.1 Split turns and tool continuity

Normally a compaction cut sits at a turn boundary. A turn begins with a user message and includes the assistant’s response and resulting tool calls until the next user message. Pi avoids cutting between an assistant tool call and its tool result because that would make the remaining context internally incoherent for the provider.

An exceptionally large turn can exceed the recent-history budget by itself. The compaction design has a split-turn path that summarizes the early prefix and preserves the later portion. This is why an extension should not assume compaction always removes only whole, old user turns.

7.8 Branch navigation and branch summaries

`/tree` changes the leaf to a different entry. The old branch may contain work that is relevant to the new branch but is no longer on its parent chain. Pi can summarize that abandoned path and append a `branch_summary` at the navigation point. That summary becomes context on the new path.

```
old path:    common ancestor -> A -> B -> C
new target:  common ancestor -> D -> E
```

```
optional summary of A/B/C is appended near E so the new conversation has
a concise account of the abandoned work without replaying all old messages.
```

`branchSummary.reserveTokens` controls the summarizer's response reserve. `branchSummary.skipPrompt` controls whether the interactive UI asks before making a summary. Extensions can provide/cancel summaries with `session_before_tree` and observe them with `session_tree`.

7.9 Session replacement is not a mutation of the old runtime

New/resume/fork flows use `AgentSessionRuntime`. It owns both the current `AgentSession` and cwd-bound services such as settings, resources, models, and auth. A replacement flow deliberately tears down the old runtime and creates a new one for the destination session.

```
request /resume or extension command
-> session_before_switch (may cancel)
-> session_shutdown on old extension runtime
-> old AgentSession disposed
-> services/session created for target cwd
-> extension runtime rebound
-> session_start on the new extension instance
-> optional withSession callback receives fresh context
```

An extension command can call `ctx.newSession`, `ctx.switchSession`, or `ctx.fork`. Its `withSession` callback receives a fresh replacement context. Captured old `pi`, command context, or `SessionManager` objects are stale after the replacement and must not be reused. Capture plain strings/IDs before the switch, then use only the callback context for session-bound operations.

7.10 Session names, labels, and human navigation

`/name` and `pi.setSessionName()` append `session_info`. The selector then shows the stored name rather than deriving a name from the first user message. Labels are per-entry bookmarks, added by `pi.setLabel(entryId, label)`, and appear in tree navigation. They are lightweight metadata: setting a label does not edit a historic message or change model context.

Use labels for checkpoints, handoff points, and experiment names. Use a session name for the overall task. This division makes a large session tree easier to scan without placing extra prose in provider context.

7.11 Extension persistence patterns

7.11.1 Stateful tool results

When state is the consequence of a tool call, return a serializable snapshot in `details` and reconstruct from current-branch tool results in `session_start`. This keeps state branch-aware.

```
pi.on("session_start", (_event, ctx) => {
  for (const entry of ctx.sessionManager.getBranch()) {
    if (entry.type !== "message" || entry.message.role !== "toolResult") continue;
    if (entry.message.toolName !== "project_notes") continue;
    // Rebuild extension state from entry.message.details after validating it.
  }
});
```

7.11.2 Private durable data

Use `pi.appendEntry("my-extension", data)` when the model does not need the state. On next `session_start`, scan `getEntries()` or the branch for custom entries with that `customType`. Register an entry renderer if users should see the data in the transcript.

7.11.3 Persistent model context

Use `pi.sendMessage({ customType, content, display })` when the message should be retained *and* given to the model. Register a message renderer if the default transcript presentation is insufficient. Do not encode secrets or verbose debug data in a custom message because it becomes conversation context.

7.12 Session debugging checklist

When a resumed session behaves unexpectedly, answer these in order:

1. Which JSONL file is active (`getSessionFile()`)?
2. Which leaf is selected (`getLeafId()`)?
3. Which entries are in `buildContextEntries()`?
4. Is a compaction or branch summary affecting context?
5. Which model/thinking-level entry is last on the full path?
6. Is the extension reading the current branch or every historic entry?
7. Did a replacement flow leave a stale object captured in a closure?

This approach is faster than treating session persistence as an opaque feature: the entire model context is explainable as a deterministic view of the active JSONL tree.

8 Resources, settings, and project trust

Before the first prompt, Pi assembles a runtime from configuration, credentials, models, extensions, skills, prompts, themes, and context files. This chapter explains that startup surface so you can change configuration intentionally and know when a project-local change will be ignored.

Primary implementation files are `packages/coding-agent/src/core/settings-manager.ts`, `src/core/resource-loader.ts`, `src/core/trust-manager.ts`, and `src/core/agent-session-services.ts`. The user-facing configuration reference is `packages/coding-agent/docs/settings.md`.

8.1 The startup services boundary

`createAgentSessionServices()` creates the cwd-bound services needed for a session:

```
cwd + agent directory
-> SettingsManager
-> AuthStorage
-> ModelRegistry
-> DefaultResourceLoader
    -> reload resources subject to trust
-> apply pending provider registrations from extension factories
```

This separation is deliberate. A resumed session can have a different cwd than the process's initial cwd, so settings, project resources, and models must be created for the effective session cwd. The `AgentSessionRuntime` rebuilds this service bundle on a session replacement rather than pretending that settings from the old project apply to the new one.

8.2 Global and project settings

Pi reads JSON settings from two conventional scopes:

File	Scope	Relative resource paths resolve from
<code>~/.pi/agent/settings.json</code>	All projects for the current user	<code>~/.pi/agent</code>
<code>.pi/settings.json</code>	The current project	<code>.pi</code>

Project settings override global settings. Nested objects are merged, so a project can adjust only one compaction field while retaining the global default for the rest:

```
// global
{ "compaction": { "enabled": true, "reserveTokens": 16384 } }

// project
{ "compaction": { "reserveTokens": 8192 } }
```

The effective configuration has `enabled: true` and `reserveTokens: 8192`. Use this pattern for small project adjustments rather than copying a full settings file that becomes stale as Pi adds defaults.

8.3 Trust is an execution policy, not a cosmetic prompt

Project-local settings can declare extensions/packages and project skills can instruct a model to execute commands. Pi therefore does not automatically run project resources in an unknown folder.

Interactive startup checks whether a project contains trust-requiring resources and whether a saved decision applies to that folder or a parent. In the absence of a decision it can ask. Trust enables project `.pi/settings.json`, project resources, missing project package installation, and project extension code.

Noninteractive modes cannot ask. They follow global `defaultProjectTrust`:

Value	Noninteractive behavior without saved decision
"ask"	Ignore project resources because there is no UI prompt
"never"	Ignore project resources
"always"	Trust project resources

`--approve/-a` and `--no-approve/-na` override trust for one run. `/trust` saves a decision for future starts but does not retroactively reload the current runtime; restart or use the appropriate workflow after changing trust.

Extensions receive `project_trust` before project-local extensions are loaded. Only global/user or CLI-provided extensions are eligible at this point. A handler returns `trusted: "yes" | "no" | "undecided"`; the first yes/no result owns the decision. This prevents a project extension from approving its own execution before Pi has decided to load it.

8.4 Resource categories

The resource loader assembles several independently useful kinds of content:

Resource	What it changes	Typical discovery location
Extension	Runtime behavior, tools, commands, UI, providers	<code>extensions/</code> , configured paths
Skill	Reusable instruction bundle and optional <code>/skill:name</code> command	<code>skills/</code> , <code>.agents/skills</code>
Prompt template	User-invoked prompt expansion	<code>prompts/</code> , configured paths

Resource	What it changes	Typical discovery location
Theme	Terminal styling data	themes/ , configured paths
Context file	Instructions added to system-prompt construction	AGENTS.md and related loader results
Package	Bundle containing the above	npm/git/local source

These are not interchangeable. A skill is model instruction; an extension is executable code; a theme is presentation. If a customization only needs to tell the model how to work, use a skill/prompt/context file rather than an extension with full process permissions.

8.5 Explicit resource paths and patterns

The settings keys **extensions**, **skills**, **prompts**, and **themes** accept local file paths or directories. **packages** adds package sources. Arrays support glob patterns plus filtering syntax:

Syntax	Meaning
pattern	Include matching paths
!pattern	Exclude matching paths
+path	Force-include this exact package-relative path
-path	Force-exclude this exact package-relative path

Keep path resolution in mind when moving configuration between scopes: a relative global path and a relative project path have different base directories. Use absolute paths or `~` only when the resource truly belongs to one machine, not to a repository.

8.6 Skills and prompt templates on the input path

`AgentSession.prompt()` gives extension commands first chance to handle slash input. It then emits the extension `input` hook. If input is not handled, Pi expands skill commands and prompt templates. That means a raw `/skill:name` is not necessarily the message sent to the model; it can become the skill's content plus user arguments.

`enableSkillCommands` controls whether skills are registered as commands. A skill can still be loaded as system-prompt resource even if its direct command surface is disabled, depending on how it is supplied. When debugging prompt content, inspect both `ctx.getSystemPromptOptions()` and the actual context event rather than searching only the typed text.

8.7 Context files and system-prompt construction

The loader provides loaded context files and skills to `AgentSession`. `_rebuildSystemPrompt()` creates structured `BuildSystemPromptOptions` from:

- `cwd`;

- selected tool names, snippets, and guidelines;
- custom/append system-prompt CLI options;
- loaded skills;
- loaded context files.

`buildSystemPrompt()` turns those options into provider-independent system prompt text. The system prompt is rebuilt when the active tool set changes and when session resources reload. Extensions that replace the prompt in `before_agent_start` receive the already-built prompt and can chain a change, but should avoid discarding these inputs unintentionally.

8.8 Settings that alter runtime behavior

Settings are more than UI preferences. The following groups affect runtime semantics and should be considered part of a reproducible project setup.

8.8.1 Model and thinking

`defaultProvider`, `defaultModel`, `defaultThinkingLevel`, `enabledModels`, and `thinkingBudgets` influence startup selection, model cycling, and provider options. A saved session model takes precedence during a resume if it is still configured and authenticated; defaults are a fallback.

8.8.2 Compaction and branch summary

`compaction.enabled`, `reserveTokens`, and `keepRecentTokens` determine when and how much conversation is summarized. `branchSummary.reserveTokens` and `skipPrompt` affect tree navigation. These settings change what reaches a provider, so they are not merely transcript retention preferences.

8.8.3 Retry and delivery

The `retry` block controls agent-level retry and provider request settings. Provider retries default to zero; agent-level retry is separate. Keeping them distinct avoids hiding a long provider retry delay from the session-level logic.

`steeringMode` and `followUpMode` select whether queued messages drain all at once or one at a time. `transport`, `httpIdleTimeoutMs`, and `websocketConnectTimeoutMs` influence provider transport behavior.

8.8.4 Shell, images, and terminal

`shellPath`, `shellCommandPrefix`, and `npmCommand` affect process execution and package management. Image settings control `resize/block` behavior before a provider request. Terminal image and rendering settings affect display but do not make an image model-capable; model input capabilities still control whether an image can be sent.

8.9 Project configuration that an extension should respect

An extension with its own project-level configuration should:

1. Use `CONFIG_DIR_NAME` from coding-agent rather than hardcoding `.pi`.
2. Resolve the path relative to `ctx.cwd`.
3. Call `ctx.isProjectTrusted()` before reading that configuration.
4. Validate parsed JSON as untrusted data.
5. Define which state is global, project-local, and session-local.

For example, a global extension may store user preferences in the agent directory but read a project allowlist only after trust. That matches Pi's own security model and avoids creating an unreviewed bypass.

8.10 Packages are resource delivery, not a new execution mode

Pi packages distribute extensions, skills, prompts, and themes from npm, git, or a local path. A package can declare a `pi` manifest in `package.json` or use conventional subdirectories. The package manager records sources in settings; the resource loader then finds the declared resources.

Package scope follows settings scope:

- Global installs are user-wide and live under the agent directory.
- Project installs are stored under `.pi` and are trust-gated.
- A local path is loaded in place, so edits affect the next load/reload.

`pi install`, `pi remove`, `pi list`, and `pi update --extensions` operate on this package layer. They do not change the fact that an extension is regular code with the privileges of the Pi process.

8.11 Themes and the TUI boundary

Theme paths load JSON themes. Themes influence colors/formatting used by interactive components and extension renderers; they do not modify keybindings or component logic. An extension can query/change themes through `ctx.ui`, but should use semantic theme keys (`accent`, `warning`, `toolTitle`, and so on) instead of ANSI constants so the user's theme continues to work.

8.12 Reload semantics

`/reload` reloads extensions, skills, prompts, themes, and context files. It does not preserve arbitrary extension closures. The documented lifecycle is:

```
session_shutdown (old extension runtime)
-> resources reloaded
-> session_start { reason: "reload" } (new runtime)
-> resources_discover { reason: "reload" }
```

Code after `await ctx.reload()` still executes in the old command call frame. Treat reload as terminal for that handler. Rebuild all session-scoped state in the new `session_start` path.

8.13 Configuration-review checklist

Before committing a project `.pi/settings.json` or package declaration, check:

1. Does this setting execute code or fetch/install dependencies?
2. Does the team intend to trust the repository for that behavior?
3. Are environment references used for secrets instead of literal credentials?
4. Are relative paths intentional for this scope?
5. Does a nested override preserve the global fields you expect?
6. Will noninteractive CI receive the required trust decision?
7. Can the change be represented as a skill/theme/template instead of code?

These questions make configuration review an architectural step rather than an afterthought.

9 Built-in tools: filesystem and shell execution

The coding agent ships seven named tools: `read`, `bash`, `edit`, `write`, `grep`, `find`, and `ls`. They are not special to the agent loop. Each is a TypeBox-schema-backed tool definition wrapped into an `AgentTool`, which is why extensions can observe, replace, or add tools through the same lifecycle.

Implementation lives in `packages/coding-agent/src/core/tools/`. The public construction helpers are re-exported from the coding-agent package. This chapter focuses on behavior that matters when using, extending, or debugging the tools, not on teaching shell commands.

9.1 Tool registry and default selection

`core/tools/index.ts` is the tool factory map. It defines the `ToolName` union and dispatches each name to a `create*ToolDefinition` or `create*Tool` helper.

```
tool definition: schema + description + execute + optional renderers
    |
    v
wrapToolDefinition(...)
    |
    v
agent-core AgentTool: validated arguments + execution callback
```

The coding-agent SDK creates all definitions so extensions can see metadata, but the initial active set is the four coding tools: `read`, `bash`, `edit`, and `write`. `grep`, `find`, and `ls` are available definitions but not part of that default active list. Explicit tool selection, `--no-builtin-tools`, extensions, or `pi.setActiveTools()` can change what the model receives.

9.2 Shared design principles

Every built-in tool follows the same broad contract:

1. A TypeBox schema validates the model's arguments in `agent-loop.ts`.
2. `execute` receives an ID, parsed params, an optional abort signal, and an optional incremental-update callback.
3. Success returns model-facing content plus optional structured `details`.
4. Failure throws; the agent loop converts it into an error tool result.
5. Optional renderers turn call/result data into a compact TUI row.

Tool descriptions and `promptGuidelines` are not decoration. Coding-agent uses active tools' snippets/guidelines while rebuilding the system prompt. For example, `read` tells the model to inspect files with `read` rather than `cat` or `sed`; `write` says it is for new files or complete rewrites.

9.3 Output limits protect the next provider turn

Large tool output can consume the same context window needed for reasoning and response. `truncate.ts` defines a shared 50 KB / 2,000 line limit, along with head/tail/line helpers and metadata describing why output was truncated.

The correct truncation direction depends on the tool:

Tool use	Useful part	Typical policy
File content or search result	Beginning/ordered matches	Keep head
Long-running command log	Latest state/error	Keep tail
Individual match line	Prefix is usually enough	Truncate line

Built-in tools report truncation to the model and provide an actionable next step. A custom tool must do the same. Quietly dropping output makes a model repeat a command or assume a partial result is complete.

9.4 read: text ranges and image attachments

`read.ts` accepts a required `path` plus optional 1-indexed `offset` and `limit`. It resolves the path relative to the session cwd, checks readability, then branches on MIME detection.

For text files it splits lines, applies the requested range, and then applies shared truncation. The continuation messages are deliberately specific:

```
Showing lines 1-2000 of 6500. Use offset=2001 to continue.
```

If a single first line exceeds the byte limit, it reports that condition and suggests a bounded shell fallback. That avoids pretending the complete line was read.

For supported images (JPEG, PNG, GIF, WebP, BMP), it reads binary data, processes/resizes it subject to settings, and returns an image content block alongside a text note. The tool checks the current model's input capabilities and adds a note when the model cannot receive images. The global `images.blockImages` setting adds another later defense in the conversion path.

9.4.1 Read operations are pluggable

`ReadOperations` has `readFile`, `access`, and optional MIME detection. The default uses the local filesystem. An extension can build a read tool backed by SSH, a container, or another remote system by providing those operations. The schema and TUI behavior stay the same; only the I/O boundary changes.

That is preferable to reimplementing the full read tool merely to redirect filesystem access.

9.5 write: create or replace a complete file

`write.ts` accepts `path` and full `content`. It creates parent directories recursively, overwrites existing content, and returns a byte count. Its prompt guideline explicitly reserves it for new files or complete rewrites.

The implementation wraps the complete operation in `withFileMutationQueue`:

```
resolve target path
-> wait for same-file queue
-> check abort
-> mkdir parent
-> check abort
-> write file
-> check abort
-> release queue
```

The queue is keyed by a resolved, canonical path. Existing paths are passed through `realpath`, so symlink aliases share a queue; missing target paths use the resolved absolute path. Different files still execute in parallel.

Notice the abort design: the write tool does not reject from an abort-event listener and release the queue while a filesystem operation is still in flight. It checks `signal.aborted` between awaited operations, keeping the queue held until the current operation has settled. A custom mutating tool should follow the same model.

9.6 edit: exact replacement and queued mutation

`edit.ts` is for targeted changes to an existing file. Its schema is built around a path and edit blocks with exact old/new text. The exact-match approach lets Pi detect ambiguous or stale replacements rather than applying a fuzzy patch to the wrong location.

Like `write`, `edit` resolves the target and wraps the full read-modify-write window in `withFileMutationQueue`. Queue only the final write and two parallel tools can both read old content, derive conflicting replacements, then lose one change. The lock must cover reading, validation, transformation, and write.

Use `edit` rather than `write` when preserving unrelated content is important. Use `write` when creating a file or deliberately replacing its entirety. The model prompt guidelines encode this distinction to reduce destructive rewrites.

9.7 bash: process execution with streamed tail output

`bash.ts` accepts a command and optional timeout in seconds. It uses `createLocalBashOperations()` by default, which resolves the configured shell, verifies the cwd, starts a child process with stdout/stderr pipes, tracks a detached process tree where the platform supports it, and kills that tree for cancellation or timeout.

The tool can apply a configured `shellCommandPrefix` and an extension-provided `spawnHook` that changes command, cwd, or environment. This is the intended seam for sandbox/container/remote routing; it avoids a brittle string rewrite inside a generic `tool_call` hook.

9.7.1 Streaming output

`OutputAccumulator` collects stdout and stderr. While a process runs, `bash` throttles partial `onUpdate` events to roughly one every 100 ms. The TUI can show a live preview while the model waits for final output.

Final command output keeps the tail. If output was truncated, the tool saves the full output to a temporary file and includes that path in `details` and a model-facing notice. Nonzero exit status, timeout, and abort become thrown errors with accumulated output appended where available. The agent loop then returns that failure as an error tool result to the model.

This is why a failed command can still be useful: the model receives the log and exit status rather than merely an exception string.

9.8 `grep`, `find`, and `ls`: bounded repository discovery

These search/listing tools use the cwd and respect ignore policy. Their schemas make the intended scope explicit:

Tool	Important inputs	Default limit	Result shape
<code>grep</code>	pattern, optional path/glob, literal/case/context options	100 matches	File path and line-numbered matching lines
<code>find</code>	glob pattern, optional path	1,000 results	Matching paths relative to search directory
<code>ls</code>	optional directory path	500 entries	Alphabetical listing, directory suffix /, includes dotfiles

All three also use the shared byte cap. `grep` additionally limits long output lines. The tools are better suited to agent context than raw `rg/find` shell output because their limits and formatting are predictable.

They are exposed as built-in names and can be registered/activated by an extension. If you override one, retain its result details and renderer contract unless you intentionally own every affected display/session behavior.

9.9 Paths, cwd, and remote operations

Tools must distinguish what the model typed from the actual target path. The built-ins resolve relative paths against the session cwd, not an arbitrary process cwd, and format paths for display relative to the session when useful.

Remote execution does not mean every tool must become shell text. The exported operations interfaces make I/O replaceable per tool:

<code>ReadOperations</code>	<code>readFile + access + optional MIME detection</code>
<code>WriteOperations</code>	<code>mkdir + writeFile</code>
<code>EditOperations</code>	<code>read/write/mkdir-style file mutation operations</code>
<code>BashOperations</code>	<code>exec with data callback, cwd, signal, timeout, env</code>
<code>Grep/Find/Ls</code>	<code>operation interfaces for filesystem/search behavior</code>

An extension can create a tool with remote operations, register it under the same name, and leave Pi's validation/rendering behavior intact. The extension documentation's SSH and sandbox examples demonstrate this pattern.

9.10 User shell commands are separate from model bash

In interactive mode, input beginning with `!` runs a user shell command; `!!` marks its result excluded from LLM context. The extension `user_bash` event can provide custom operations or a complete result for that path.

That is not identical to an assistant's `bash` tool call. The user is directly requesting execution, while a tool call is model-generated and is represented in the assistant/tool-result protocol. A sandbox extension that wants consistent behavior often handles both: it supplies operations for built-in tools and for `user_bash`.

9.11 Tool rendering is a first-class interface

Built-in definitions include `renderCall` and `renderResult` functions. The interactive tool row receives current arguments, partial/final state, expanded flag, prior component, execution time, and an `invalidate()` callback. This permits syntax-highlighted read/write previews, compact default rendering with an expand hint, streamed shell output, and error-specific presentation.

An extension that overrides built-in execution can omit a renderer and inherit the built-in renderer per slot. That convenience depends on keeping the expected call args/result details shape. If an override changes those shapes, provide compatible renderers or a self-contained rendering shell.

9.12 Tool lifecycle and extension interception

The agent loop emits this sequence for each tool call:

```
tool_execution_start
-> schema validation and tool_call extension gate
-> execute / tool_execution_update ...
-> tool_result extension patch
-> tool_execution_end
-> final toolResult message events
```

In default parallel mode, start/preflight is source-ordered, execution/update events can interleave, and final tool-result messages are emitted in assistant source order. Do not build a custom tool that assumes another tool's result is already in `ctx.sessionManager` during its own `tool_call` handler.

9.13 A safe custom file-mutation skeleton

The following is a shape, not a complete product tool. It shows the required ordering for an extension that changes one file.

```
import { withFileMutationQueue } from "@earendil-works/pi-coding-agent";
import { readFile, writeFile } from "node:fs/promises";
import { resolve } from "node:path";

async function replaceOnce(cwd: string, path: string, oldText: string, newText: string, signal?: AbortSignal) {
  const target = resolve(cwd, path);
  return withFileMutationQueue(target, async () => {
    if (signal?.aborted) throw new Error("Operation aborted");
    const current = await readFile(target, "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");
    if (!current.includes(oldText)) throw new Error("Expected text was not found");
    await writeFile(target, current.replace(oldText, newText), "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");
  });
}
```

In a real tool, add a TypeBox schema, precise description, result content, truncation where relevant, and tests for no-match, abort, and concurrent calls.

9.14 Tool-author review questions

Before exposing a custom tool to a model, answer:

1. Does the TypeBox schema constrain the actual safe input shape?
2. Is the tool active only when its prompt guidance is appropriate?

3. Does every large output have a bounded, actionable representation?
4. Does cancellation reach all long-running work?
5. Are writes serialized across aliases of the same target?
6. Does a thrown error give the model enough detail to recover?
7. Are **details** durable and renderer-compatible?
8. Does the behavior work in parallel with built-in tools?
9. Is a policy gate better implemented as `tool_call` interception than inside the tool itself?

Treat tools as a public protocol between the model, session, UI, and extension ecosystem—not as a private helper function.

10 Models, providers, streaming, and authentication

Pi separates four concerns that are often conflated in smaller applications:

```
model metadata    -> what a model can do and how much it costs
provider config   -> where/how requests are sent
credentials       -> how a provider is authenticated
API adapter       -> how normalized Pi context becomes provider protocol
```

The separation lets the coding agent present one model selector and one agent loop while supporting OpenAI-compatible servers, Anthropic, Google, Bedrock, OAuth, proxies, and extension-defined providers.

Key source files are `packages/ai/src/types.ts`, `src/compat.ts`, `src/models.ts`, `packages/coding-agent/src/core/model-registry.ts`, and `src/core/auth-storage.ts`.

10.1 The Model contract

A model is a data record consumed by the agent and provider layer. Its important fields include provider identifier, model ID, API type, input capabilities, reasoning/thinking support, context window, maximum output tokens, cost, base URL, headers, and compatibility flags.

The model's `api` field chooses an adapter such as `openai-completions`, `openai-responses`, `anthropic-messages`, `google-generative-ai`, or `bedrock-converse-stream`. The provider name is not enough: two provider names can use the same wire protocol, and one provider can expose models with different API requirements.

Use `input: ["text", "image"]` only when a model can actually receive image content. That metadata drives UI/tool behavior and protects against sending an image to a text-only model.

10.2 Model registry assembly

`ModelRegistry` begins with generated built-in provider/model data, applies `models.json` provider and per-model overrides, merges custom models, applies OAuth-provider model modifications where required, and finally applies dynamic extension provider registrations.

```
built-in provider catalogs
-> models.json provider overrides
-> models.json per-model overrides
-> custom models merged/upserted
-> OAuth model adjustment
-> extension registerProvider configuration
-> current ModelRegistry.models
```

The registry preserves built-in models even when `models.json` is invalid; it records the load error for the interactive application to display. That is a useful resilience property: a typo in a local model file should not make all known providers disappear.

`refresh()` resets dynamic API/OAuth registrations, reloads disk config, then reapplies registered extension providers. This prevents stale registrations from surviving a model configuration reload.

10.3 Selecting an initial model

At session construction, `createAgentSession()` first tries the model recorded in a nonempty session, then consults scoped/default models through `findInitialModel()`. It restores or chooses a thinking level and clamps it to the selected model's capability. A missing configured model becomes a helpful fallback message rather than an unexplained null choice.

The selection is persisted with `model_change` and `thinking_level_change` entries. Resume is therefore not simply “use today's global default”; it attempts to continue the model choice that created historical context.

10.4 Thinking levels are Pi vocabulary

Pi's model-facing thinking levels are `off`, `minimal`, `low`, `medium`, `high`, `xhigh`, and `max`. Providers use different APIs and vocabulary. A model's `thinkingLevelMap` maps Pi levels to provider-specific values and can set a level to `null` when unsupported.

```
{
  "id": "custom-reasoner",
  "reasoning": true,
  "thinkingLevelMap": {
    "minimal": null,
    "low": null,
    "medium": null,
    "high": "default",
    "xhigh": null,
    "max": "maximum"
  }
}
```

This is more precise than a boolean **reasoning** flag. A selector can hide unsupported levels, and the session can clamp a requested level instead of serializing a value the upstream provider does not understand.

10.5 Custom models in `models.json`

The conventional file is `~/.pi/agent/models.json`. A minimal local OpenAI-compatible provider looks like this:

```
{
  "providers": {
    "local": {
      "baseUrl": "http://localhost:11434/v1",
      "api": "openai-completions",
      "apiKey": "local-placeholder",
      "models": [{ "id": "local-coder" }]
    }
  }
}
```

The placeholder key is deliberate for local services that ignore it. Pi uses configured-auth status when deciding whether to offer a model, so a provider with no auth source would otherwise be unavailable even if its server accepts unauthenticated requests.

A full model definition can set human name, **reasoning**, inputs, context window, maximum tokens, cost rates, and compatibility behavior. Cost rates are per million tokens; tiers can replace the whole request's rate set above an input threshold. This metadata affects footer/session accounting, not the provider's bill itself.

10.6 Provider overrides and model overrides

There are two different ways to change a built-in provider:

Change	Configuration	Effect
Route existing models through a proxy	provider <code>baseUrl/headers</code> , no <code>models</code>	Keep built-in catalog, change request configuration
Add/replace custom models	provider <code>models</code> array	Merge/upsert with built-ins for <code>models.json</code> behavior
Adjust a known model only	<code>modelOverrides</code> keyed by model ID	Preserve catalog, change selected metadata
Replace dynamic extension provider catalog	<code>pi.registerProvider(..., { models })</code>	Replace models for that dynamic provider

The last distinction is easy to miss: the extension `registerProvider` API treats supplied `models` as a replacement for that provider's current models, while `models.json` can merge custom models into a built-in provider. Use the mechanism whose semantics you intend.

10.7 Config values and secrets

Provider `apiKey` and header values can be literals, `$ENV_VAR/ ${ENV_VAR}` interpolation, or a leading `!command` whose stdout becomes the value. `$$` escapes a dollar and `$!` escapes a leading bang.

```
{
  "apiKey": "$TEAM_PROVIDER_KEY",
  "headers": {
    "x-tenant": "$TENANT_ID",
    "x-token": "!security find-generic-password -ws team-provider"
  }
}
```

Command values resolve at request time. Pi intentionally does not guess a cache/TTL policy for arbitrary credential commands. If a command is costly or rate limited, provide a wrapper with the caching semantics your environment needs. Do not assume checking model availability runs the command; auth-status checks deliberately avoid executing command-backed values.

10.8 Credential precedence and storage

`AuthStorage` persists API-key and OAuth credentials in `auth.json` under the agent directory. Its file backend creates parent directories with restrictive permissions and writes the file mode as `0600`. It uses a lock around reads and writes so multiple Pi processes do not race during token refresh.

Credentials may come from several places. The registry first asks `AuthStorage` for a provider key without its broad fallback, then uses configured provider API-key values when available. Runtime `--api-key` overrides are in memory only. Provider-scoped environment values can override process environment for config expansion.

The result is not merely a string. `getApiKeyAndHeaders()` produces either an error or an API key, merged provider/model headers, and provider-scoped environment values. If `authHeader: true` is configured, the registry adds an Authorization Bearer header from the resolved API key and fails clearly when it is absent.

10.9 Provider headers and request layering

Request headers are layered in practical order:

1. Model-level static headers.
2. Resolved provider headers from configuration.
3. Resolved model-specific request headers.
4. Coding-agent attribution/session/request headers.
5. Extension `before_provider_headers` mutations.
6. Provider adapter defaults and transport-specific handling.

Later sources can override earlier sources; a null header value can suppress a provider default where the adapter supports it. Avoid logging this merged map: it can contain secrets as well as useful tracing IDs.

10.10 The unified streaming API

`packages/ai/src/types.ts` defines a provider-independent `Context` and an `AssistantMessageEventStream`. `StreamOptions` carries abort signal, key, transport preference, cache retention, session ID, headers, timeout, retry, payload/response hooks, metadata, and provider-scoped environment.

`streamSimple(model, context, options)` in `src/compat.ts` is a dispatch point:

```
model selected by coding-agent
-> built-in model dispatch, or API provider lookup by model.api
-> provider.streamSimple(model, Context, options)
-> normalized AssistantMessageEventStream
```

Individual API adapters serialize messages/tools/system prompt into their wire format, parse streaming chunks, and emit normalized partial/final events. The agent loop therefore does not need a branch for every vendor protocol.

10.11 Payload and response hooks

`StreamOptions` has `onPayload` and `onResponse`. Coding-agent connects these to extension events. `before_provider_request` sees the provider-specific payload after an adapter serializes Pi context. `after_provider_response` sees status/headers after HTTP response receipt and before body consumption when the provider exposes them.

Use these for narrow instrumentation or compatibility experiments. A broad payload rewrite is fragile because it takes ownership of a provider protocol. If the endpoint genuinely differs from supported APIs, use a custom provider stream implementation instead.

10.12 Transport, timeout, and retry

Pi's shared `Transport` values are `sse`, `websocket`, `websocket-cached`, and `auto`. Providers that do not support a choice ignore it. HTTP idle timeout, WebSocket connection timeout, provider SDK retry count, and max retry delay are separate settings because they govern different failures.

The coding-agent stream function applies settings and calls `streamSimple` with an effective timeout. It treats a configured zero HTTP idle timeout as an effectively disabled finite timeout for SDKs that interpret zero as immediate expiry. Agent-level retry happens later in `AgentSession`, after it receives a final failed assistant message, and is separate from SDK/provider retries.

10.13 Custom providers from an extension

`pi.registerProvider(name, config)` can override endpoint/headers for an existing provider without model replacement, register a new provider with model definitions, register OAuth callbacks for `/login`, or register a custom `streamSimple` implementation for a nonstandard API.

Provider config validation rejects incoherent registration: models require a base URL and either API-key config or OAuth; each model must resolve an API type. An async extension factory is the correct place for remote model discovery because Pi waits for it before startup/model listing continues.

When unregistering, the registry refreshes and reapplies remaining registered providers so built-in models/protocol behavior previously overridden by that provider are restored.

10.14 Provider debugging workflow

When a model does not appear or a request fails, work in this order:

1. Inspect `ModelRegistry.getError()` for a `models.json` parse/load error.
2. Confirm provider/model ID and API type.
3. Check `getProviderAuthStatus()` rather than printing secrets.
4. Confirm `baseUrl`, `compat`, input capabilities, and thinking map.
5. Use `before_provider_request` only to inspect a minimal sanitized payload.
6. Check `after_provider_response` status/headers where supported.
7. Distinguish provider error/timeout from agent-level retry/compaction behavior.

This preserves Pi's architecture: the model registry owns configuration and credentials; the adapter owns protocol translation; the agent owns turn flow.

11 Agent core: runs, events, queues, and tool batches

`packages/agent` is the reusable runtime beneath the coding agent. It knows about a model, a system prompt, messages, tools, streaming, queues, and cancellation. It deliberately does not know about JSONL sessions, themes, project trust, extensions, or built-in filesystem tools.

The central source files are `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.

11.1 The three layers inside the package

Layer	Main file	Responsibility
Data contracts	<code>types.ts</code>	Tool/message/event/loop configuration types
Stateful controller	<code>agent.ts</code>	Current state, queues, aborting, listeners, lifecycle
Pure-ish loop orchestration	<code>agent-loop.ts</code>	Provider turn, event emission, tool batches, continuation

This split is useful when debugging. If state appears stale, start at **Agent**. If a provider turn or tool batch sequence is wrong, start at `agent-loop`. If the caller cannot supply a valid hook/tool/message shape, start in `types.ts`.

11.2 Agent state and snapshots

The **Agent** holds state including:

- current system prompt, model, thinking level, and active tools;
- durable messages accumulated in this runtime;
- streaming message and pending tool IDs for UI observers;
- active run/abort controller;
- steering and follow-up message queues;
- event listeners and optional caller hooks.

Before a prompt starts, it creates a context snapshot with copies of the messages and tool arrays. The snapshot is shallow with respect to each message, but protects the loop from a caller mutating the arrays themselves during a turn. The loop owns its current context object and appends new messages as it emits events.

11.3 Starting a prompt versus continuing a context

`Agent.prompt()` normalizes input to one or more `AgentMessage` values. It rejects a concurrent active run, then calls `runAgentLoop()` with the snapshot. `Agent.continue()` instead uses the current transcript and requires its last message not to be an assistant message, except for explicit queued-message handling paths.

The distinction prevents accidental replay:

<code>prompt(messages)</code>	adds new user/custom messages and begins a run
<code>continue()</code>	asks the model again from existing context
<code>steer(message)</code>	queues content between assistant turns
<code>followUp(message)</code>	queues content after the agent would otherwise stop

Coding-agent uses this deliberately. It starts with a user message, then may call `continue()` after automatic retry/compaction preparation or messages queued by a post-run extension event.

11.4 Active-run lifecycle and cancellation

`runWithLifecycle()` owns one `AbortController` per run. It marks the state as streaming, calls the loop, converts an unexpected thrown failure into an assistant error/aborted message sequence, then clears the active run.

`Agent.abort()` aborts that controller. The signal is passed to the provider stream and tool execution. A tool/extension that starts child work must forward the signal itself; abort does not magically cancel a detached fetch/process whose API was never given the signal.

`waitForIdle()` resolves after the active run and awaited event listeners finish. In coding-agent, true application idleness can also include session policy after the low-level loop, such as retry or compaction. This is why extension authors should use the higher-level `agent_settled` event when they need Pi rather than only the current loop to be done.

11.5 Event stream as the observable protocol

The loop emits events rather than updating a UI directly. Important event families are:

Events	Meaning
<code>agent_start</code> , <code>agent_end</code>	One low-level run begins/ends
<code>turn_start</code> , <code>turn_end</code>	A provider response plus any tool batch
<code>message_start</code> , <code>message_update</code> , <code>message_end</code>	User, assistant, and tool-result message lifetime
<code>tool_execution_start</code> , <code>tool_execution_update</code> , <code>tool_execution_end</code>	Tool row/process lifetime

`Agent.processEvents()` first updates internal state, then awaits registered listeners. This ordering means a listener sees a coherent agent state through the event it is handling. Coding-agent relies on that property when syncing session entries and calling extension hooks.

11.6 One provider turn in detail

`streamAssistantResponse()` is the provider boundary in `agent-loop.ts`.

```
AgentMessage[] context
-> optional transformContext
-> convertToLlm
-> provider Context { systemPrompt, messages, tools }
-> streamFn(model, context, options)
-> for await normalized provider events
-> partial/final AssistantMessage plus message events
```

The low-level loop does not assume a provider has the same message shape as an agent message. `convertToLlm` belongs to the caller precisely because a coding agent may have session-only messages while another embedding has a different message representation.

On a provider `start` event, the loop pushes a partial assistant message into its current context and emits `message_start`. Text, thinking, and tool-call deltas replace that partial state and emit `message_update`. A final `done` or `error` resolves the stream result and emits `message_end`.

11.7 Tool argument preparation and validation

After a final assistant message, the loop finds content blocks with `type: "toolCall"`. It resolves the requested tool from active context, calls optional `prepareArguments`, then validates the result against the tool schema.

`prepareArguments` exists for session compatibility. An extension/tool can adapt legacy stored call shapes into its current strict schema before validation without weakening the public parameters declaration. It is not a general way to accept arbitrary invalid input.

An unknown tool, invalid schema, hook failure, or explicit block becomes an immediate error result rather than a crash. The model receives that error in a tool-result message and can decide how to recover.

11.8 Sequential and parallel tool execution

`ToolExecutionMode` has two choices:

Mode	Behavior
<code>sequential</code>	Prepare, execute, finalize, and persist each tool before the next
<code>parallel</code>	Preflight calls in source order; run prepared tools concurrently; finalize completion events as they finish; emit final result messages in source order

The **Agent** default is **parallel**. Parallel does not mean all events arrive in arbitrary order: preflight remains deterministic. It does mean a tool hook cannot assume a sibling's result has finished or been persisted. Use queues or turn-end state when tools need coordination.

Some tool calls can request sequential execution through their definition or the batch's execution policy. Inspect the active tool behavior rather than assuming a single global mode if you build an advanced extension.

11.9 Before and after tool hooks

The generic loop accepts two caller callbacks:

- **beforeToolCall** receives assistant message, tool call, validated args, and current context. It may block with a reason.
- **afterToolCall** receives the executed result/error state and may replace content, details, error flag, or terminate hint.

Coding-agent connects these callbacks to **ExtensionRunner.emitToolCall** and **emitToolResult**. That bridge keeps the reusable agent package free of extension types while allowing extension hooks to participate in execution.

11.10 Termination, steering, and follow-up

When a tool batch completes, the loop creates final **toolResult** messages and appends them to context. Unless every result has **terminate: true**, it performs another provider turn. The all-results rule avoids one tool silently stopping a batch whose other tools expect a model follow-up.

After each completed turn, the loop asks for steering messages. Steering messages enter before the next provider request. When there are no more tool calls and no steering messages, it asks for follow-up messages. Follow-ups restart the inner loop only after the agent would otherwise stop.

```
assistant/tool batch complete
-> steering queue? yes: inject before next model call
-> no tools/steering: follow-up queue? yes: begin another turn
-> neither: agent_end
```

This behavior is the technical meaning behind interactive Enter vs Alt+Enter while Pi is running.

11.11 Next-turn refresh hook

The loop can call **prepareNextTurn** after **turn_end**. It may replace context, model, or thinking level for the next provider call. Coding-agent uses this to refresh the current system prompt and active tool list from session state.

This matters when an extension dynamically changes tools or model state during a run. The current turn remains internally consistent; the next provider call sees the refreshed session snapshot rather than a stale configuration.

11.12 Error semantics are intentionally normalized

The `StreamFn` contract says ordinary request/model/runtime failures should be represented by a returned assistant event stream whose final message has `stopReason`: `"error"` or `"aborted"`, rather than a rejected promise. The agent also protects itself by converting unexpected throws into a final assistant error message.

Tool failures are different: `execute` throws become an error `toolResult`, so a model can read output/error and choose a recovery. This difference is important for tests and UI: a provider failure ends the run; a tool failure usually lets the model continue.

11.13 Embedding the agent package

If you use `packages/agent` outside coding-agent, you must provide policy that coding-agent normally supplies:

- initial `AgentState` with a valid model/system prompt/tools;
- `convertToLlm` for your message representation;
- provider `streamFn` and credentials or key resolver;
- listeners that persist/render state if needed;
- context transformation/queue behavior appropriate to your application;
- tool definitions and any permission gate.

The package gives a rigorous loop, not a complete coding application. That is the reason Pi can support the interactive CLI, SDK examples, and other hosts without coupling all of them to terminal/session decisions.

11.14 Agent-core debugging checklist

When a turn misbehaves, capture these facts before editing code:

1. What did `convertToLlm` produce?
2. What system prompt and active tools were in the context snapshot?
3. Which normalized provider events arrived, and was a final message emitted?
4. Did the assistant end with tool calls, error, aborted, length, or stop?
5. Which tool arguments passed validation/preparation?
6. Was the batch sequential or parallel, and what event ordering occurred?
7. Did steering/follow-up queues inject another message?
8. Did a `prepareNextTurn` refresh change model/tools/context?

This reduces a complex agent issue to a small number of observable protocol boundaries.

12 Terminal UI and interactive mode

Pi's interactive application is built from a reusable terminal UI package and a coding-agent-specific composition layer. The separation is worth learning: an extension normally customizes the coding-agent UI surface, while a contribution to `packages/tui` changes primitives that could serve another terminal application.

Relevant files include `packages/tui/src/tui.ts`, `src/terminal.ts`, `src/keybindings.ts`, `src/autocomplete.ts`, and `packages/coding-agent/src/modes/interactive/interactive-mode.ts`.

12.1 The TUI package owns terminal mechanics

The public barrel `packages/tui/src/index.ts` exports the TUI class, component/container interfaces, text/markdown/editor/list components, terminal implementations, key parsing/matching, keybindings, autocomplete, image support, and ANSI-aware text utilities.

The TUI class is responsible for the component tree, focus, layout, input routing, overlays, terminal lifecycle, and differential screen updates. A component returns lines for an available width and can invalidate itself when its rendered state changes. This is a retained component model, not immediate `console.log` output.

12.2 Differential rendering in practical terms

Terminal rendering is expensive and visually disruptive if every update clears the whole screen. The TUI tracks prior lines and writes only the changes needed to transform the terminal into the newly rendered view.

```
component tree changes
-> render visible lines with ANSI styling
-> compare with previous terminal frame
-> emit minimal cursor movement/text changes
```

That design enables token-by-token assistant updates, tool-output streaming, and autocomplete without a full redraw for every byte. It also explains why components should invalidate only when necessary and reuse component instances for high-frequency partial updates where the API supports it.

12.3 Components are composition, not application policy

The TUI exports general components such as `Text`, `Box`, `Container`, `Markdown`, `Editor`, `Input`, `SelectList`, `SettingsList`, `Loader`, and `Image`. Their job is layout/presentation/input behavior. They do not know what a session, provider, or tool result means.

Coding-agent builds a chat-specific component tree around those primitives:

```
InteractiveMode
-> header/status regions
-> chat transcript container
    -> user/assistant/thinking/tool components
-> pending messages/tool area
-> widgets above/below editor
-> editor or extension replacement editor
-> footer and overlays/dialogs
```

This means a custom transcript renderer should return a **TUI Component**; it does not need to manage raw terminal escape sequences itself.

12.4 Terminal lifecycle and capabilities

`ProcessTerminal` and terminal support code manage `stdin/stdout`, raw mode, size information, terminal capabilities, and cleanup. Image rendering is capability-dependent: Pi can choose Kitty, iTerm2, or fallback behavior based on what the terminal supports.

Interactive mode initializes terminal/theme behavior before starting its event loop and stops/restores terminal state at shutdown. If you add a feature that opens an external editor, subprocess, or custom modal, follow the existing interactive-mode lifecycle rather than leaving raw terminal state altered.

12.5 Keyboard input is parsed before it becomes a binding

`keys.ts` parses terminal input into key semantics and `keybindings.ts` maps semantic keybinding IDs to user-configurable keys. A binding ID is stable, namespaced policy such as:

```
tui.input.submit
tui.editor.cursorWordLeft
tui.select.confirm
app.interrupt
app.tools.expand
app.model.cycleForward
```

An extension should display hints with `keyHint("app.tools.expand", "to expand")` or inspect the injected `KeybindingsManager`. It should not hardcode raw `ctrl+x` checks. Hardcoding bypasses user configuration and breaks the mapping between the keybindings file, help text, and actual behavior.

12.6 Editor submission and queue semantics

The interactive editor is not just a text box. Its submission handler handles:

- built-in slash commands;
- `!/!!` user shell commands;
- compaction-time message holding;
- steering/follow-up delivery while an agent is active;
- normal queued input for the main `run()` loop;
- editor history, multiline text, image paste, autocomplete, and external edit.

The high-level outcome is simple: an idle normal submission resolves `getUserInput()`, whose loop calls `session.prompt(text)`. While streaming, regular Enter sends a steering prompt; Alt+Enter sends a follow-up prompt. The detailed handling stays in `InteractiveMode` so `AgentSession` can serve print/RPC clients without importing terminal components.

12.7 Transcript rendering from agent events

`InteractiveMode.subscribeToAgent()` receives coding-agent session events. `handleEvent()` dispatches behavior by type:

Event	UI responsibility
<code>agent_start</code>	Clear pending tool bookkeeping and begin activity state
<code>message_start</code>	Create/update the appropriate transcript row
<code>message_update</code>	Refresh partial assistant/thinking/tool-call display
<code>message_end</code>	Finalize transcript component and session-visible output
tool execution events	Create/update/complete a tool row
<code>turn_end</code> / <code>agent_end</code>	Final activity/footer/pending-message updates

The UI works from events rather than polling the model. This preserves the same visual behavior whether an assistant emits text, thinking, tool calls, errors, or a synthetic retry/compaction message.

12.8 Tools in the transcript

Tool rows are composed by the interactive components around a tool definition's `renderCall` and `renderResult` slots. The context passed to those renderers includes current args, a row-local mutable state object, optional last component, partial/final flags, expanded state, show-images preference, cwd, timing, and an `invalidate()` method.

This produces a useful layering:


```

tool definition knows how to render its call/result data
interactive mode knows where the row belongs in chat and when it updates
TUI knows how to layout/diff-render the resulting components

```

For an extension tool, return a compact default component and make details available when expanded. For a high-frequency partial result, reuse `context.lastComponent` if safe instead of creating a new large component on every stream update.

12.9 Extension UI surface

`ctx.ui` is an adapter owned by `InteractiveMode`. It gives extensions a controlled surface rather than direct access to private chat/editor fields:

API family	Examples
Dialogs	<code>select</code> , <code>confirm</code> , <code>input</code> , <code>editor</code>
Notices/status	<code>notify</code> , <code>setStatus</code> , working message/indicator controls
Editor/widgets	<code>setEditorText</code> , <code>pasteToEditor</code> , <code>setWidget</code> , custom editor
Transcript/rendering	message/entry renderers, tool expansion controls
Look and completion	themes, autocomplete provider, custom footer
Complex interaction	<code>custom(factory)</code> and experimental overlays

Use `ctx.hasUI` for methods that require user interaction and `ctx.mode === "tui"` before terminal-only behavior. In print/JSON modes, a dialog cannot block for input. In RPC mode, Pi can map some UI operations to protocol events but custom terminal components are not an equivalent surface.

12.10 Custom components and focus

`ctx.ui.custom()` temporarily replaces the editor with a factory that receives the TUI instance, current theme, a keybindings manager, and a `done(value)` callback. The component handles input until it calls `done`.

```

const answer = await ctx.ui.custom<boolean>((_tui, _theme, keybindings, done) => {
  const component = new Text("Enter confirms; Escape cancels", 1, 1);
  component.onKey = (key) => {
    if (keybindings.matches(key, "tui.select.confirm")) done(true);
    if (keybindings.matches(key, "tui.select.cancel")) done(false);
    return true;
  };
  return component;
});

```

The example uses named bindings, so a user who rebinds `confirm/cancel` receives the expected behavior. More complex overlays have focus/visibility APIs; use them only when an editor replacement/dialog cannot express the interaction.

12.11 Autocomplete composition

The TUI supplies autocomplete interfaces and a combined provider. Pi's interactive editor layers slash-command and path completion. An extension can add a provider by receiving the current provider and delegating when its own trigger syntax does not apply.

The reliable pattern is:

1. Inspect text before the cursor.
2. If extension syntax matches, return an explicit prefix and candidate list.
3. Otherwise call the previous provider.
4. Delegate completion application unless custom insertion is necessary.
5. Preserve existing file-completion decisions unless deliberately changing it.

This prevents an extension's `#issue` or `$variable` completion from disabling normal slash/path completion elsewhere in the line.

12.12 Themes and semantic styling

Theme functions provide semantic styles such as `accent`, `muted`, `dim`, `success`, `warning`, `error`, and `toolTitle`. Use `theme.fg(name, text)` plus style helpers like `theme.bold(text)` in renderer code.

Do not embed fixed ANSI colors in an extension renderer. A literal color may be unreadable in a user's light/custom theme and cannot participate in Pi's visual language. Semantic keys let an extension look native across themes.

12.13 Images and rendered width

Terminal width is measured in display cells, not JavaScript string length. Wide CJK characters, ANSI escape sequences, hyperlinks, and image rows make naive `text.length` calculations wrong. The TUI exports utilities such as `visibleWidth`, `truncateToWidth`, and ANSI-aware wrapping. Use them in custom components rather than slicing strings manually.

Likewise, terminal image support is capability- and cell-size-dependent. Use the TUI `Image` component/terminal image helpers rather than trying to print a provider image's base64 data as text.

12.14 Debugging visual issues

When an interactive feature is wrong, isolate its layer:

1. Capture the agent/session event sequence; verify data before rendering.
2. Inspect the tool/message renderer output as a component.
3. Check available width and ANSI-aware visible width.
4. Confirm the component invalidates after its state changes.
5. Test narrow terminal widths and long/CJK/ANSI-rich content.
6. Test focus and cancel paths for custom UI.
7. Use `/debug` for rendered TUI lines and the last model messages.

The correct fix is usually at the narrowest layer that owns the defect: a tool renderer for tool presentation, interactive mode for chat composition, or the TUI package for a general terminal layout/input primitive.

13 Testing, debugging, and contribution workflow

Architecture knowledge becomes useful when it shortens a safe debugging loop. Pi’s repository rules distinguish code checks, targeted tests, interactive testing, and real-provider behavior. Follow the narrowest useful validation path; do not make a provider call merely to prove a local state transition.

The authoritative local rules are in `AGENTS.md`. Coding-agent also documents development behavior in `packages/coding-agent/docs/development.md` and extension behavior in `packages/coding-agent/docs/extensions.md`.

13.1 Start with a reproducible boundary

For a report such as “Pi ignored my extension” or “a tool result rendered wrong,” identify which boundary is wrong before changing code:

Symptom	First boundary to inspect
Extension is absent	Trust decision, resource loader discovery, factory load error
Command runs but prompt is unchanged	Extension command/input/before-agent event path
Model gets wrong messages	Session context, <code>context</code> hook, <code>convertToLlm</code> , provider payload
Tool has invalid args	Schema, <code>prepareArguments</code> , model tool-call block
Tool output is lost	Truncation, tool result, parallel ordering, renderer
Provider call fails	Model registry/auth, adapter response, retry configuration
UI is stale or clipped	Session event, renderer component, TUI invalidation/width
Resume/fork loses state	Session branch/context, extension reconstruction, stale replacement closure

This map prevents a common waste of time: editing the TUI when the agent never emitted a message event, or editing a provider adapter when an extension never loaded due to project trust.

13.2 Static checking versus tests

After code changes, repository policy calls for `npm run check`. It formats and checks the workspace, verifies pinned dependencies/import rules/shrinkwrap state, and runs the root TypeScript check. It is a broad code-quality gate, not the test suite.

Tests prove behavioral expectations. Do not use `npm run build` as a proxy for tests, and do not run the full Vitest suite casually: the repository warns that some end-to-end tests can activate when `provider/auth` environment variables are present. For broad non-LLM coverage, use `./test.sh` from the repository root; for a targeted coding-agent test, run the Vitest CLI from the package root as described by `AGENTS.md`.

13.3 Test at the correct package

Change area	First test location/type
Provider serialization/stream parsing	<code>packages/ai/test/</code> unit/regression tests
Agent tool scheduling/message events	<code>packages/agent</code> tests or harness tests
TUI width, key, component behavior	<code>packages/tui/test/</code>
Coding-agent sessions/config/extensions	<code>packages/coding-agent/test/</code>
Agent behavior without a paid provider	<code>packages/coding-agent/test/suite/</code> with faux provider

The coding-agent suite harness is intentional. For tests under `packages/coding-agent/test/suite/`, use `test/suite/harness.ts` and the faux provider. Real model keys introduce cost, nondeterminism, rate limits, and a different failure surface than the behavior being tested.

13.4 High-value test cases by subsystem

13.4.1 Extensions

- Discovery from global/project/CLI paths and trust gating.
- Async factory completion before provider/model use.
- Correct event ordering and return semantics.
- Tool block/allow behavior for policy hooks.
- Reload cleanup and state reconstruction.
- Mode fallback when `ctx.hasUI` is false.
- Session replacement with a fresh `withSession` context.

13.4.2 Tools

- Schema validation and legacy `prepareArguments` shape.
- Success, thrown error, abort, and timeout.
- Bounded output and continuation/full-output notices.
- Parallel sibling behavior and same-file mutation queue.
- Renderer partial/final/expanded behavior.

13.4.3 Sessions and compaction

- JSONL loading/migration and malformed input.
- Tree path selection and label/session-info changes.
- Context rebuild with compaction and branch summaries.
- Model/thinking restoration.
- Resume/fork/new runtime cleanup and extension lifecycle.

13.4.4 Provider/model registry

- Custom models/overrides merge semantics.
- Auth source precedence and command/env configuration.
- Header merging and `authHeader` behavior.
- Dynamic register/unregister restoration.
- Compatibility flags for a protocol-specific regression.

13.5 Interactive TUI smoke testing

`AGENTS.md` documents a controlled tmux pattern for testing the interactive application. The important principle is to use a detached terminal with known dimensions, capture the pane after startup, send text/keys, then cleanly kill the session. This produces a repeatable artifact for editor, command, streaming, and rendering behavior without relying on the developer's current terminal state.

For a UI reproduction, record terminal dimensions, theme and keybinding overrides, pasted text/image input, exact keys, model/streaming state, and captured terminal lines or a `/debug` log. That information often explains a visual issue that a screenshot alone cannot reproduce.

13.6 `/debug` and diagnostic data

The hidden `/debug` command writes `~/.pi/agent/pi-debug.log` with rendered TUI lines containing ANSI codes and the last messages sent to the LLM. It is a source of truth for a rendering/context mismatch, but it may contain sensitive prompts, file contents, and tool results. Treat it as local sensitive data; do not paste it into a public issue unreviewed.

Extension diagnostics should be equally disciplined. Log an event name, safe IDs, timing, and sanitized state—not API keys, auth headers, complete provider payloads, or full context-file content.

13.7 Error classification before retrying

Pi has several retry-like mechanisms with different ownership:

Failure/condition	Owner	Typical action
Invalid tool input or tool execution error	Agent loop/model	Return error tool result; model may recover

Failure/condition	Owner	Typical action
Transport/provider transient error	AgentSession/provider settings	Retry according configured policy
Context overflow	AgentSession compaction policy	Compact and retry if applicable
User abort	Active run abort controller	Stop abort-aware work; do not auto-retry blindly
Extension hook error	Extension runner	Log; tool-call errors fail safe by blocking
Project not trusted	Trust manager	Do not load project code/resources

Misclassifying these leads to bad fixes. Retrying an invalid schema will not correct model arguments; compacting a missing API key is pointless.

13.8 Source navigation habits

The repository is large enough that broad search results can mislead. A useful sequence is to find a public symbol with `rg`, read its contract and focused call sites, identify the package boundary crossed, and then read a nearby test or example. Change one boundary at a time and add a regression where needed.

Examples are valuable for extensions. Start with `packages/coding-agent/examples/extensions/hello.ts`, then choose a focused example such as `permission-gate.ts`, `dynamic-tools.ts`, `entry-renderer.ts`, or `custom-provider-*` instead of copying a large multi-feature extension.

13.9 Git hygiene in a shared worktree

The repository may have concurrent sessions changing unrelated files. Stage only explicit paths you changed; never use broad staging commands. Inspect `git status` before staging/committing, and do not reset, clean, stash, or checkout unrelated work. The project rules also prohibit commits unless the user asks.

For generated model metadata, change the generator source rather than editing `packages/ai/src/models.generated` by hand. That policy preserves the link between catalog data and its generation process.

13.10 Documentation as a tested interface

This guide is source-grounded documentation. Keep its code examples subject to the same review questions as implementation:

1. Does every exported symbol exist at the documented package path?
2. Does the example respect actual `async/cancellation` semantics?
3. Does a lifecycle claim match the runner/agent implementation?
4. Does a configuration example state merge/replace/trust semantics?

5. Does a visual description name the actual renderer/component boundary?

When an API changes, update its focused chapter and the prompt lifecycle chapter together. The lifecycle is the integration test for the book: it reveals contradictions between isolated component explanations.

13.11 Pre-review checklist

Before requesting review for a code or extension change:

1. Reproduce the original behavior with a minimal scenario.
2. State the package and public contract that own the fix.
3. Add/run the narrowest regression test or controlled interactive proof.
4. Run the repository-required static check for code changes.
5. Check trust/security implications of new configuration or package loading.
6. Test cancellation, error, and reload/replacement paths where relevant.
7. Inspect diffs for generated/lockfile/dependency changes you did not intend.

This workflow makes Pi's extension power manageable: every change is tied to a specific runtime boundary and an evidence-producing validation path.

14 Guided source tours and architecture exercises

The earlier chapters describe Pi's architecture. This chapter turns that map into repeatable reading exercises. Each tour has a question, an entry point, the important control-flow boundary, and a concrete outcome. Work through them with the source open; the purpose is to learn how the modules connect, not to memorize file names.

14.1 Tour 1: What launches when pi starts?

Question: Which module turns process arguments into an interactive agent?

Start at `packages/coding-agent/src/cli.ts`. It sets process title/environment, configures the HTTP dispatcher early, and calls `main(process.argv.slice(2))`. Then follow `main.ts`.

Trace these checkpoints:

1. `parseArgs` turns raw argv into a typed argument object.
2. `resolveAppMode` selects RPC, JSON, print, or interactive behavior from explicit flags and TTY state.
3. Startup settings/trust/session inputs are prepared.
4. `createAgentSessionRuntime` creates services and a session.
5. `InteractiveMode`, `runRpcMode`, or `runPrintMode` receives that runtime.

Outcome: You should be able to answer why a piped stdin invocation may use print mode and why interactive mode can keep the caller's cwd while resolving Pi's own assets from its package configuration.

14.2 Tour 2: Why can a session change cwd safely?

Question: What is recreated when `/resume` opens a session from another working directory?

Start at `core/agent-session-runtime.ts`. Read `switchSession`, then the shared teardown/apply helpers. Follow the `createRuntime` factory passed from `main.ts`.

Draw this before reading implementation:

```
old session + old cwd services
-> session shutdown/dispose
-> SessionManager.open(target)
-> target cwd services
-> target AgentSession
-> rebind interactive UI
```

Check which objects are cwd-bound: `SettingsManager`, `DefaultResourceLoader`, and the runtime's session/model configuration all depend on the effective cwd. The shared global auth storage can remain reusable, but its resolution is still consulted by the new model registry.

Outcome: You should understand why extension code must not reuse an old session context after a replacement and why `withSession` exists.

14.3 Tour 3: How do resources reach a system prompt?

Question: Where do `AGENTS.md`, skills, prompts, and tool guidance join?

Start at `core/resource-loader.ts`, then find callers of `resourceLoader.getSystemPrompt`, `getSkills`, and `getAgentsFiles` in `core/agent-session.ts`. Read `_rebuildSystemPrompt` and then `core/system-prompt.ts`.

Write the data as a structured object before it becomes text:

```
cwd
context files
skills
custom/append prompt
selected tools
tool snippets
tool guidelines
  -> BuildSystemPromptOptions -> string
```

Now inspect `setActiveToolsByName`. It rebuilds the prompt because a changed tool selection must change the instructions available to the model.

Outcome: You can explain why adding a tool has both a runtime registry effect and a prompt/context effect.

14.4 Tour 4: What happens to a user message before a provider sees it?

Question: Which transformations can change user text?

Read `AgentSession.prompt` top to bottom. List each return/branch in order:

1. Extension command resolution.
2. `input` event and possible transform/handled result.
3. Skill/template expansion.
4. Streaming steer/follow-up queue behavior.
5. Auth/model checks and possible compaction.
6. User/custom message construction.
7. `before_agent_start` message/system-prompt modifications.
8. `_runAgentPrompt`.

Then read the `Agent` call path and `streamAssistantResponse` to see the later context transform plus `convertToLlm` boundary.

Outcome: You can choose `input`, `before_agent_start`, `context`, or `before_provider_request` based on the representation you need to change.

14.5 Tour 5: Follow one streamed token to the terminal

Question: How does a provider delta become a changed terminal row?

Start in `packages/agent/src/agent-loop.ts` at `streamAssistantResponse`. Find how `text_delta`, `thinking_delta`, and `toolcall_delta` replace the partial assistant message and emit `message_update`. Follow the event callback to `Agent.processEvents` in `agent.ts`, then to coding-agent's session and interactive subscribers.

Finally find `InteractiveMode.handleEvent`. Do not try to read the entire file linearly; search for the event type and the component method it calls.

Outcome: You can locate a rendering bug as one of four categories: provider event normalization, agent partial state, session event bridge, or interactive component update.

14.6 Tour 6: Follow one tool call through validation

Question: Why cannot an extension tool execute arbitrary model JSON?

Start at `executeToolCalls` in `agent-loop.ts`. Follow one call through:

```
assistant content block
  -> active tool lookup
  -> prepareArguments
  -> validateToolArguments
  -> beforeToolCall
  -> execute
  -> afterToolCall
  -> ToolResultMessage
```

Next open `packages/coding-agent/examples/extensions/hello.ts` and identify where its Type-Box schema becomes both a static parameter type and a runtime validator. Compare it with `core/tools/read.ts`, which adds operations and renderers but retains the same fundamental definition shape.

Outcome: You can explain why a schema belongs in an extension even when TypeScript already knows the desired params type.

14.7 Tour 7: Study parallel tool execution without guessing

Question: What ordering is guaranteed when an assistant asks for two tools?

Read `executeToolCallsParallel` and contrast it with the sequential path. Write a timeline for two calls A and B:

```
start(A), validate/gate(A)
start(B), validate/gate(B)
execute(A) and execute(B) concurrently
end events in completion order
final ToolResultMessage(A), ToolResultMessage(B) in source order
```

Compare that with the extension documentation’s tool-execution guarantees. Then inspect `withFileMutationQueue` to see how same-file writes gain a narrower serialization guarantee without making unrelated tools sequential.

Outcome: You can design an extension tool that works correctly under the default mode rather than accidentally relying on timing.

14.8 Tour 8: Explain a model request without opening a network trace

Question: Where do key, headers, model metadata, and payload come from?

Start at the stream function in `core/sdk.ts`. It asks the model registry for auth, combines environment/header settings, emits `before_provider_headers`, then calls `streamSimple`.

Follow into `packages/ai/src/compat.ts`. It chooses a built-in model dispatcher or resolves the API provider by `model.api`. Open one adapter such as `api/openai-completions.ts` or `api/anthropic-messages.ts` and identify its serialization and event-stream parser entry points.

Outcome: You can distinguish configuration bugs (registry), protocol bugs (adapter), and runtime-loop bugs (agent) without a speculative rewrite.

14.9 Tour 9: Trace a resume-context discrepancy

Question: Why did a resumed model refer to an old branch or summary?

Open `core/session-manager.ts` and read `buildContextEntries` plus `buildSessionContext`. Use a small hand-drawn tree with a compaction entry and a branch summary. Trace the active leaf backwards, then identify which entries become context messages and which are metadata only.

Cross-check with `docs/session-format.md`. Use `getBranch`, `buildContextEntries`, and `buildSessionContext` from an extension command to inspect an actual session instead of parsing JSONL manually at first.

Outcome: You can prove whether a context difference is persistence policy, compaction, branch navigation, or an extension’s reconstruction mistake.

14.10 Tour 10: Add a narrowly scoped extension feature

Question: How do you add a feature without coupling to internals?

Pick one small behavior such as a status widget, a custom command, or a read-only tool. Start from the closest example. Name the lifecycle event/API that owns it, then write a one-file extension under a global auto-discovery directory. Verify startup, action, reload, and noninteractive behavior.

Use this decision table:

Desired result	Narrow public seam
Ask model to use a new operation	<code>registerTool</code>
Let user invoke a task	<code>registerCommand</code>
Block/mutate a tool call	<code>on("tool_call")</code>
Add context per user prompt	<code>on("before_agent_start")</code>
Change provider endpoint/models	<code>registerProvider</code>
Store private session state	<code>appendEntry</code>
Render a custom persistent record	<code>registerEntryRenderer</code>
Add a small always-visible indicator	<code>ctx.ui.setStatus</code> or <code>widget</code>

Outcome: You practice the extension API as a set of explicit seams rather than as permission to reach into `InteractiveMode` or `AgentSession` private state.

14.11 A weekly reading routine for new contributors

Large codebases become legible through repeated, bounded questions. A practical routine is:

Session	Focus	Deliverable to yourself
1	CLI startup and services	One startup diagram with package boundaries
2	Agent loop and messages	One provider/tool turn timeline
3	Sessions/compaction	One active-branch/context sketch
4	Built-in tools	One tool contract comparison
5	Extensions	A working one-file extension and reload proof
6	TUI	A captured interactive event/render trace
7	Provider config	A local/custom provider configuration review

The guide is intentionally organized so each exercise has a corresponding chapter. Revisit the prompt lifecycle after every tour: it is the shortest integration model of the entire project.

15 Extension event and API reference

This chapter is a practical reference for choosing extension hooks. It does not replace the source types in `packages/coding-agent/src/core/extensions/types.ts` or the exhaustive `packages/coding-agent/docs/extensions.md`; use those when writing code. Its goal is to make event choice, return value, context, and lifecycle safety explicit before implementation.

15.1 Choose an event by the representation you need

The same user action appears in several representations. Select the latest boundary that still contains the information you need, because later hooks are more specific and less likely to disturb unrelated behavior.

Need to inspect/change	Event/API	Representation
Raw submitted text before skills/templates	<code>input</code>	Text/images plus source/delivery mode
Prompt-specific system context	<code>before_agent_start</code>	Expanded prompt and system-prompt options
Message history before every LLM call	<code>context</code>	Deep-copied agent messages
Final HTTP headers	<code>before_provider_headers</code>	Mutable header map
Provider wire payload	<code>before_provider_request</code>	Adapter-specific payload
Provider status/response headers	<code>after_provider_response</code>	HTTP metadata before stream consumption
Validated tool arguments	<code>tool_call</code>	Tool name and mutable input
Tool execution output	<code>tool_result</code>	Content/details/error patch opportunity
User typed ! command	<code>user_bash</code>	User command and context-exclusion choice
Durable session operation	Session events or command context	Session metadata/tree/replacement methods

The rows describe different capabilities, not aliases. For example, a context filter cannot reliably rewrite a provider's serialized protocol payload, and a provider hook cannot alter Pi's own system-prompt string reported to other extensions.

15.2 Startup and trust events

15.2.1 project_trust

This event occurs before Pi decides whether project-local dynamic resources may run. It is visible only to user/global and CLI-provided extensions, not project extensions. The handler must return a trust decision:

```
pi.on("project_trust", async (event, ctx) => {
  if (!ctx.hasUI) return { trusted: "undecided" };
  const yes = await ctx.ui.confirm("Trust project?", event.cwd);
  return yes ? { trusted: "yes", remember: true } : { trusted: "undecided" };
});
```

Return "undecided" when your extension does not own the policy. A yes/no decision suppresses later handlers and the built-in flow. Only use **remember** when it is appropriate to persist a global trust decision for this user.

15.2.2 resources_discover

This fires after **session_start** so an extension can return additional skill, prompt, and theme paths. It receives **reason**: "startup" | "reload". It is a discovery hook, not an instruction to read arbitrary project paths without checking trust. Use the surrounding extension/session context's trust state for project-local behavior.

15.3 Session events

15.3.1 session_start and session_shutdown

Use **session_start** to initialize state that requires an active session: restore durable data, add UI status, start a watcher, or open a connection. Its reason identifies startup, reload, new, resume, or fork. Use **session_shutdown** to close that same resource. Cleanup must be idempotent because reload, replacement, and quit may all trigger it.

Do not start long-lived resources in the factory. A factory may run for a command that never creates a session, while session events make lifecycle ownership explicit.

15.3.2 session_before_switch and session_before_fork

These are cancellation gates. A switch handler sees whether the operation is **new** or **resume** plus an optional target path. A fork handler sees entry ID and position (**before** for fork or **at** for clone). Return { **cancel**: true } to block a user action after a confirmation/policy check.

Do not call a session-changing method from this ordinary event handler. If an extension must initiate a switch, use a command handler with command-context APIs described later in this chapter.

15.3.3 Compaction and tree hooks

`session_before_compact` can cancel compaction or return a custom summary plus the required kept-entry/token metadata. `session_compact` observes the durable result and whether it came from an extension. `session_before_tree` can cancel or provide a branch summary; `session_tree` observes the selected leaf and summary outcome.

Use these hooks for policy/summary customization. Use `context` if the goal is only to hide/filter data for a provider call; a context hook does not change the durable compaction/tree semantics.

15.3.4 Session information

`session_info_changed` fires after `/name` or `pi.setSessionName`. It is a good place to update a title/status integration. It is not a model message and does not require rebuilding the system prompt.

15.4 Input and agent-run events

15.4.1 input

Input handlers run after extension command lookup and before skill/template expansion. The result controls the next step:

Result	Effect
no result / <code>continue</code>	Leave input unchanged
<code>transform</code> with text/images	Pass modified input into later expansion
<code>handled</code>	Stop input processing; no agent run starts

Transforms chain in extension load order. Keep transforms simple and make source-aware decisions with `event.source` (`interactive`, `rpc`, or `extension`). During streaming, `event.streamingBehavior` distinguishes steer and follow-up queue delivery.

15.4.2 before_agent_start

This runs after normal input expansion but before the low-level agent loop. It is the best prompt-scoped seam. A result may add a custom persistent message and/or replace the chained system prompt. Later handlers receive system-prompt changes from earlier handlers.

The event exposes `systemPromptOptions`, including custom prompt, selected tools, snippets/guidelines, cwd, context files, and skills. Treat it as sensitive extension-local data. It may contain full instructions that should not be copied into command descriptions or telemetry.

15.4.3 `agent_start`, `agent_end`, and `agent_settled`

`agent_start` and `agent_end` correspond to one low-level agent loop. The end does not prove application idleness: coding-agent may retry, compact/retry, or start queued continuations. `agent_settled` means no such automatic work remains. Use settled for a status integration that must not clear itself too early.

15.4.4 `turn_start`, `turn_end`, and `messages`

A turn is one assistant response plus its tool calls/results. `turn_end` includes the assistant message and final tool results. Message events give fine-grained lifecycle: user/assistant/tool-result messages emit start/end; assistant streaming also emits update events. `message_end` can replace the finalized message while preserving its role.

Message replacement is powerful. Preserve required fields and do not change a role merely to repurpose a transcript; provider conversion and session logic rely on role semantics.

15.5 Provider events

15.5.1 `before_provider_headers`

Pi assembles outgoing headers, then gives handlers a mutable map. Assign a string to add/override, or null to delete. The hook runs once per provider request; retries reuse the same headers rather than invoking it again.

Use it for a stable request correlation header. Do not use it for a credential refresh protocol that must run anew on every retry unless you explicitly own that behavior.

15.5.2 `before_provider_request`

This sees the adapter-specific payload after Pi has converted context/system prompt/tools. Return `undefined` to keep it or another value to replace it for later handlers and the actual request. The event is suitable for a narrow serialization compatibility patch or a sanitized development trace.

Because it is provider-specific, code here is coupled to API type. Guard logic by model/provider when necessary, avoid assuming every adapter uses the same JSON shape, and test against the provider's expected protocol.

15.5.3 `after_provider_response`

This fires after the HTTP response arrives and before its body stream is consumed. It receives normalized status and headers when available. Some provider transports abstract away headers; absence is a provider limitation, not an extension failure.

15.6 Tool events and parallelism

`tool_execution_start`, `tool_execution_update`, and `tool_execution_end` describe the execution row. `tool_call` is the mutable/blocking policy hook. `tool_result` is the result-patching hook. The default parallel execution order is subtle:

```
source-order starts and preflight
-> concurrent execute/update
-> completion-order tool_result/end
-> source-order final toolResult messages
```

Therefore, a `tool_call` handler sees session state synchronized through the assistant message but not necessarily sibling tool results. Use a typed helper for built-ins:

```
import { isToolCallEventType } from "@earendil-works/pi-coding-agent";

pi.on("tool_call", (event) => {
  if (!isToolCallEventType("read", event)) return;
  // event.input.path is typed for the built-in read tool.
});
```

For a custom tool, export its `Static<typeof schema>` input and supply explicit generic parameters to the guard. Direct `event.toolName === name` narrowing is not sufficient because arbitrary custom names overlap built-in literals.

15.7 Extension context reference

All handlers receive a context with these recurring values:

Field/method	Practical meaning
<code>cwd</code>	Current session working directory
<code>mode</code> / <code>hasUI</code>	Execution surface and interaction availability
<code>ui</code>	Controlled interactive UI adapter
<code>sessionManager</code>	Read-only access to durable tree/context state
<code>modelRegistry</code> / <code>model</code>	Available/current model and authentication view
<code>signal</code>	Active agent signal during turn-related work, otherwise usually undefined
<code>isIdle</code> , <code>abort</code> , <code>hasPendingMessages</code>	Flow-control observations/actions
<code>getContextUsage</code>	Context-window usage/estimate
<code>compact</code>	Trigger compaction with callbacks
<code>getSystemPrompt</code>	Current Pi system-prompt string at this lifecycle point

The scope of `signal` is important. A command handler executed while idle does not receive an active turn signal. A `tool_result` handler usually does. Pass it to nested asynchronous work only when it is defined and relevant.

15.8 Command context reference

Command handlers receive `ExtensionCommandContext`, which adds operations that would be unsafe from arbitrary events:

Method	Safe use
<code>waitForIdle</code>	Finish automatic work before a session mutation
<code>newSession</code>	Create a replacement session, optional setup/handoff
<code>fork</code>	Fork/clone from an entry
<code>navigateTree</code>	Move to a tree point with optional summary
<code>switchSession</code>	Open another session path
<code>reload</code>	Run <code>/reload</code> lifecycle
<code>getSystemPromptOptions</code>	Inspect base prompt inputs in a user-triggered command

Replacement methods accept `withSession`. Use only the callback's context for session-bound work. The old extension instance may have already run shutdown cleanup, and captured old session objects will throw or represent old state.

15.9 Registration API quick decisions

API	Use when	Avoid when
<code>registerTool</code>	Model needs a new operation	User-only interaction is enough
<code>registerCommand</code>	User needs an explicit action	Input syntax is a natural transform
<code>registerShortcut</code>	User needs a configurable keyboard action	You can use a command/menu instead
<code>registerFlag</code>	Startup behavior needs a CLI option	State can live in settings
<code>sendMessage</code>	Model needs a custom context message	State should remain private
<code>sendUserMessage</code>	Agent should receive a genuine user message	You only need a silent state update
<code>appendEntry</code>	Durable data should not affect LLM context	You need model-visible instructions
<code>registerProvider</code>	Endpoint/models/auth/streaming change	A header-only trace is enough

15.10 Safety rules for extension authors

1. Start background resources in `session_start`, clean them in `session_shutdown`.
2. Treat project-local config as untrusted until `isProjectTrusted()` says otherwise.

3. Forward cancellation signals.
4. Serialize file mutations with the shared queue.
5. Truncate model-facing output.
6. Use mode/hasUI guards for user prompts and custom components.
7. Reconstruct persistent state from session entries.
8. Use fresh context after session replacement/reload.
9. Prefer public APIs/types over internal imports.

These rules cover the lifecycle errors that are hardest to discover from a happy-path extension demo.

16 Run modes, SDK embedding, and operational boundaries

The `pi` binary is one host for the coding-agent runtime. The same session construction and event model can also serve print, JSON, RPC, and programmatic SDK use. Understanding the modes prevents terminal assumptions from leaking into automation and helps you select an integration surface deliberately.

Start with `packages/coding-agent/src/main.ts`, `src/modes/`, and `src/core/sdk.ts`. For SDK examples, see `packages/coding-agent/examples/sdk/` and `docs/sdk.md`.

16.1 How Pi chooses a mode

`main.ts` resolves application mode from explicit arguments and terminal state:

Mode	Typical trigger	Primary host
Interactive	TTY stdin/stdout without print/RPC/JSON selection	<code>InteractiveMode</code>
Print	<code>-p</code> , non-TTY stdin/stdout, or piped prompt	<code>runPrintMode</code>
JSON	<code>--mode json</code>	JSON-oriented output path
RPC	<code>--mode rpc</code>	<code>runRpcMode</code> JSON protocol

The distinction is not cosmetic. It changes whether Pi can prompt a user, how output is emitted, and which extension UI methods are meaningful. All modes still rely on the coding-agent session/runtime instead of implementing separate provider/tool loops.

16.2 Extension mode behavior

An extension context exposes `mode` and `hasUI`:

Application surface	<code>ctx.mode</code>	<code>ctx.hasUI</code>	Design implication
Interactive terminal	<code>"tui"</code>	true	Full components, dialogs, editor behavior
RPC	<code>"rpc"</code>	true	Protocol-backed dialogs/notices; custom terminal UI is limited

Application surface	<code>ctx.mode</code>	<code>ctx.hasUI</code>	Design implication
JSON	"json"	false	Do not wait for a user prompt or render components
Print	"print"	false	Agent runs, then process exits; UI calls are no-ops/fallbacks

Guard user interaction explicitly. A policy tool that requires confirmation needs a noninteractive decision, such as blocking with a clear reason, rather than awaiting `ctx.ui.confirm()` in a mode without UI.

16.3 Print mode as a single-run host

Print mode is appropriate for scripts that want one response rather than a long-lived terminal session. `main.ts` prepares initial text/images from arguments/files/stdin, then passes that input to `runPrintMode`. The normal `AgentSession.prompt()` preprocessing remains active: extensions, context, tools, provider selection, and session policy can still participate.

An extension in print mode should avoid assuming the process will stay alive after a response. Background resources, delayed notifications, and UI-only state are poor fits. Return model-facing content or durable session data if the automation needs an observable result.

16.4 JSON mode and machine-readable automation

JSON mode emits structured activity rather than terminal chat formatting. It is useful for a caller that wants to observe events without parsing ANSI output. Keep stdout clean in extensions: logging arbitrary text to stdout can corrupt a machine-readable protocol. Prefer the documented event/UI mechanisms or stderr for controlled diagnostics when appropriate.

The same caution applies to provider payload tracing. A raw JSON dump may contain private context and may interfere with automation output. Build a sanitized, opt-in diagnostic command instead.

16.5 RPC mode as a persistent process boundary

RPC mode keeps a process alive for command/request exchange. Coding-agent has RPC types, JSONL framing, client code, and an entry point under `src/modes/rpc/`. The runtime and session still own model/tool state; RPC is a transport around that behavior, not an alternate agent implementation.

Extensions can use `ctx.hasUI` in RPC mode for operations that map to protocol UI messages, but `ctx.ui.custom()` cannot become a terminal component for a remote client. Design commands/tools with a structured noninteractive fallback when they might run through RPC.

16.6 The public SDK entry point

`createAgentSession(options?)` in `core/sdk.ts` is the composition entry point for a programmatic host. It can create defaults or accept caller-provided cwd, agent directory, model, session manager, settings manager, model registry, resource loader, scoped models, tool restrictions, custom tools, and a session start event.

The SDK does several pieces of application policy on purpose:

```
resolve cwd and agent directory
-> auth/model/settings/session services
-> resource loader (unless caller supplies one)
-> restore/choose model and thinking level
-> create Agent with coding-agent conversion/provider hooks
-> create AgentSession and extension runtime
```

The result gives an `AgentSession`, extension load result, and possible model fallback message. A host can subscribe to the session, call `prompt`, provide custom tools, inspect sessions, or embed its own UI.

16.7 Why SDK callers should not reimplement `main.ts`

It is tempting to instantiate `Agent` directly because its constructor is small. That skips coding-agent semantics including resource loading, session persistence, model restore/default selection, built-in tool definitions, extension runner integration, system-prompt building, retry, and compaction.

Use `Agent` directly only when you deliberately own all of those policies. Use `createAgentSession` when you want Pi's coding-agent behavior in a new host. Use the CLI/RPC surface when a separate process boundary is preferable.

16.8 Custom tools supplied by an SDK host

`createAgentSession` accepts custom tool definitions alongside built-in-tool selection. These tools enter the same registry/extension wrapping flow as an extension-registered tool. Give them a clear source identity, schema, durable details shape, and prompt metadata where appropriate.

An SDK host can use this for application-specific operations without requiring an extension file on disk. The tradeoff is scope: a tool supplied by one host does not automatically appear in a user's ordinary `pi` invocation.

16.9 Initial messages and attachments

The CLI's initial-message machinery can combine explicit prompt text, piped stdin, file arguments, and supported images. Interactive mode feeds the resulting message/images through the same session prompt path used after the editor submits text. This keeps extension `input/before_agent_start` behavior consistent between startup prompts and later user prompts.

When embedding, construct user content as text/image blocks and let the session/agent conversion path handle the provider representation. Do not pre-serialize provider-specific image payloads in a generic SDK host.

16.10 Operational safety boundaries

Pi runs with the operating-system permissions of its process. It does not add a built-in filesystem/process/network permission sandbox. The root README recommends containerization or sandbox patterns when stronger isolation is needed.

This has mode-specific consequences:

- A print/CI invocation can execute the same tools as an interactive one.
- Trust gates project extensions/resources, but do not sandbox global code.
- A custom provider can transmit context to its configured endpoint.
- `bash`, `write`, and extension `exec` run with host process authority.

Use OS/container boundaries for security enforcement. Use extension tool gates for workflow policy and user confirmation. Do not confuse a model instruction or a theme with a security control.

16.11 Operational observability

The runtime includes timing, diagnostics, and output-guard behavior. Startup diagnostics are collected during service/session construction and reported by the application layer. Extension load failures are treated as startup errors with a hint to retry without extensions. Settings/model load issues surface as warnings/errors appropriate to the mode.

For a production-like integration, capture:

- selected mode and cwd;
- selected model/provider/API type, without credentials;
- session ID/path and persistence state;
- extension/resource load diagnostics;
- agent/turn/tool timing and completion reason;
- provider status metadata when safely available;
- cancellation/retry/compaction decisions.

Those observations map directly to the package boundaries in this guide and make a remote incident diagnosable without collecting the full user prompt.

16.12 Choosing an integration approach

Goal	Best starting point
Human coding in a terminal	Interactive CLI
One scripted answer	Print mode
Machine-readable event/result stream	JSON mode
Long-lived external controller	RPC mode
Node application sharing Pi semantics	<code>createAgentSession</code> SDK
New model protocol/provider behavior	Provider extension or <code>packages/ai</code> contribution
Custom execution policy/UI in existing Pi	Extension

Choosing the boundary early is an architectural decision. It determines who owns terminal state, persistence, authentication, user confirmation, and process lifetime.

16.13 Operational checklist

Before putting Pi into automation or another host:

1. Choose the mode/SDK boundary intentionally.
2. Set a trust policy appropriate for noninteractive projects.
3. Select or restrict models/tools explicitly if policy requires it.
4. Store credentials in supported auth/config mechanisms, not prompt text.
5. Ensure output/logging does not corrupt JSON/RPC protocols.
6. Forward cancellation from the host into Pi's abortable work.
7. Decide session persistence and cleanup behavior.
8. Add an OS/container sandbox if untrusted code or high-risk tools are in scope.

These operational decisions are the final layer of the architecture: Pi's internal boundaries are valuable only when the host uses them deliberately.

17 Failure and recovery reference

Pi turns many failures into explicit protocol data instead of allowing an exception to escape through every layer. This reference connects symptoms to the layer that owns recovery. Use it during debugging and extension design.

17.1 Failure flow by runtime layer

```
configuration/trust failure
  -> startup diagnostic or ignored project resource

model/provider failure
  -> final assistant message: error or aborted
  -> AgentSession retry/compaction policy may decide next action

tool validation/execution failure
  -> error ToolResultMessage
  -> model receives the error and can choose a recovery action

extension hook failure
  -> runner reports error; tool_call fails safe by blocking

rendering failure
  -> fallback renderer or interactive diagnostic path
```

The different outcomes are intentional. A provider failure means there is no trusted assistant result to continue from. A tool failure is part of an assistant turn and is often useful information for the model's next request.

17.2 Startup and configuration failures

Symptom	Owning layer	Expected behavior	First recovery action
Malformed <code>settings.json</code> or <code>models.json</code>	Settings/model registry	Diagnostic/load error; built-ins remain available where possible	Correct JSON; inspect reported scope/path
Project extension unavailable	Trust/resource loader	Project resources ignored when untrusted	Save/override trust intentionally, then restart/reload

Symptom	Owning layer	Expected behavior	First recovery action
Extension factory fails	Resource loader/application	Startup diagnostics; Pi suggests retry without extensions	Run with <code>-ne</code> or <code>-e</code> to isolate path/import failure
No model selected	Session setup	Prompt is rejected before agent loop	Configure/choose model and auth
Auth missing	Model registry/session	Prompt is rejected with provider guidance	<code>/login</code> , configure key, or select authenticated model

Do not work around a trust failure by making a global extension read an untrusted project's configuration. That moves a deliberate security boundary into unreviewed custom code.

17.3 Provider failures

Provider adapters report normal failure/abort outcomes through the assistant stream contract. `Agent.runWithLifecycle()` also converts unexpected throws to a final assistant message with `stopReason: "error"` or `"aborted"`, followed by turn/agent end events. The transcript and extensions therefore see a normal terminal event sequence even when the network fails.

`AgentSession` evaluates the final assistant message. Depending on settings and error classification it may prepare an agent-level retry, auto-compact/retry after an overflow, or settle. Provider SDK retries and agent-level retries are separate knobs; an excessively long provider retry can hide useful control from the agent/session layer.

For an extension, the correct response to an `agent_end` error is normally to observe/report it, not to immediately create another competing prompt. Use `agent_settled` or a deliberate command action once the built-in policy has finished.

17.4 Tool failures

Tool processing has several failure points:

Point	Result returned to model
Tool name is not active/found	Error result naming the missing tool
<code>prepareArguments</code> or schema validation fails	Error result containing the failure reason
<code>tool_call</code> returns block	Error result with block reason
Tool execute throws	Error result with thrown error text
Abort occurs before/during work	Error result indicating cancellation
<code>tool_result</code> hook throws	Error result rather than a malformed patched result

The loop emits a tool-result message after finalization. A model can inspect the result, modify its arguments, call another tool, or explain the failure. Make tool error strings actionable: include an

invalid path, missing exact text, exit code, timeout, or next command when safe. Do not expose secrets in a tool error just because the model can read it.

17.5 Cancellation versus timeout

An abort signal expresses user/application cancellation. A timeout expresses a tool/provider policy deadline. They can lead to similar text in the transcript, but extension code should preserve the distinction:

- Forward abort signals into `fetch`, process execution, and model helpers.
- Clean up partial state deterministically when cancellation happens.
- Use a tool timeout only when the work has a clear bounded deadline.
- Do not convert user cancellation into an automatic retry.

The built-in bash tool kills its process tree for abort and timeout, collects available output, and returns an error with the appropriate status. A custom tool that launches a child process should provide equivalent cleanup rather than leaving a detached process behind.

17.6 Recovery after compaction and tree movement

Compaction/retry is recovery from context pressure, not from every provider error. A custom compaction extension must return a summary tied to the preparation's `firstKeptEntryId` and token count; an arbitrary summary string without this boundary cannot correctly rebuild session context.

Tree navigation changes the active leaf. An extension that reconstructs state from `getEntries()` instead of the active branch may accidentally recover state from an abandoned path. This can look like a failure after `/tree` even though session persistence behaved correctly.

17.7 UI and renderer fallback

Renderers are presentation code. If a custom tool/message renderer throws, Pi falls back to basic rendering for that slot where supported. Treat a fallback as a diagnostic, not evidence that result data is correct. Inspect the original content/details and renderer context, then make the renderer handle partial, error, and expanded states explicitly.

For a TUI-only issue, test the same data at a narrow terminal width and with the tool output collapsed/expanded. Many apparent rendering failures are width or ANSI-visible-width mistakes rather than agent/session failures.

17.8 Incident checklist

For a reliable bug report or regression test, record:

1. Mode, cwd, trust state, session ID/path, and selected model.
2. The input/event that triggered the problem.
3. Whether it was idle, streaming, compacting, or replacing a session.
4. The relevant final stop reason/tool result error.
5. Sanitized provider status/headers if applicable.
6. Active tools and extension/resource provenance.
7. Whether Esc/reload/new/resume changes the behavior.
8. A focused test, session tree sketch, or terminal capture proving the result.

This checklist establishes a factual boundary before a fix is proposed. It is the operational counterpart to the source tours in the previous chapter.

18 Glossary

Agent The stateful class in `packages/agent` that owns a model, system prompt, messages, active tools, queues, and one active run.

AgentSession Coding-agent policy around an **Agent**: prompt preprocessing, resources, extension bridge, system prompt, sessions, retries, and compaction.

AgentSessionRuntime Owner for one live **AgentSession** and its cwd-bound services. It replaces the runtime during new/resume/fork flows.

Assistant message event stream Async stream from `pi-ai` that normalizes provider streaming events into start, deltas, completion, and error states.

Compaction Summarizing older context into a durable session entry so recent context can fit in a model window. It is not simple transcript deletion.

Context The provider-facing bundle of system prompt, converted messages, and active tools. It is distinct from the broader persisted session.

Extension A runtime-loaded TypeScript/JavaScript factory module that receives **ExtensionAPI** and registers behavior.

Extension context The `ctx` passed to `hooks/tools/commands`, containing mode, cwd, UI, session, model registry, and potentially an abort signal.

Follow-up A queued user/custom message delivered after the agent would otherwise finish, rather than between tool turns.

Model registry Coding-agent service that knows models, providers, authentication, OAuth, and dynamic provider registration.

Resource loader Coding-agent service that discovers/loads extensions, skills, prompt templates, themes, and context files subject to trust policy.

SessionManager JSONL-tree persistence and context reconstruction API for a coding-agent session.

Steering A queued message delivered after the current assistant turn and tool batch, before the next provider request.

Tool A model-callable operation with a TypeBox parameter schema and async execute function. Built-in and extension tools share the agent-loop boundary.

Tool result A durable message produced by a tool execution. Its content goes to the model; details carry structured metadata for renderers/state.

TUI Terminal user interface. Pi's reusable TUI primitives live in `packages/tui`; coding-agent composes them into the interactive application.